

Duale Hochschule Baden-Württemberg Mannheim

Studienarbeit

Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III

Studiengang Informatik

Studienrichtung Angewandte Informatik

Verfasser:	Ertan Ertas und Christian Strerath
Matrikelnummer:	9390224, 4965013
Kurs:	TINF23AI2
Studiengangsleiter:	Prof. Dr. Holger Hofmann
Betreuer:	Prof. Dr. Karl Stroetmann
Bearbeitungszeitraum:	15.10.2025 - 14.04.2026

Erklärung

Wir versichern hiermit, dass wir unsere Studienarbeit mit dem Thema: „Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben.

Mannheim, 8. April 2026

Ertan Ertas

Kaiserslautern, 8. April 2026

Christian Strerath

Kurzfassung (Abstract)

Auf Deutsch

In English

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung	1
1.2	Zielsetzung	2
1.3	Methodik und Abgrenzung	3
2	Theoretische Grundlagen der Typisierung	5
2.1	Semantik und Funktion von Datentypen	5
2.2	Statische Typisierung mittels TypeScript	6
2.2.1	Herausforderungen einer naiven Code-Migration	8
2.2.2	Explizite Typannotationen	11
2.2.3	Strukturelles Typing und Interface-Verträge	16
2.3	Fazit zur architektonischen Neuausrichtung	23
3	RecursiveSet: API und fachliche Modellierung	25
3.1	Mathematische Mengen und Wohlunterscheidbarkeit	25
3.2	Die Datenstruktur RecursiveSet	26
3.3	Strukturierte Elemente: Die Tuple-Klasse	28
3.4	Assoziative Daten: Die RecursiveMap	29
3.5	Der Freeze-Contract: Immutabilität sichern	30
3.6	Klassische Mengenoperationen	32
3.7	Funktionale Transformationen als Komprehensions-Ersatz	35
3.7.1	Basis-Transformationen: map, filter und reduce	36
3.7.2	Komplexe Konstruktionen: Verschachtelte Iterationen auflösen mit flatMap	36
4	Vom Python-Werkzeug PLY zu Lezer	39
4.1	Grundlagen von Lezer	40
4.2	Grammatikdefinition in Lezer	42
4.3	Generierung des Parsers	43
4.4	Integration in die TypeScript-Implementierung	45
5	Case Study: Umsetzung eines vollständigen Interpreters in TypeScript	48
5.1	Ziel und Aufbau des Notebooks	48
5.2	Sprachdefinition im Vergleich: Python (PLY) vs. TypeScript (Lezer)	49
5.3	Transformation vom CST zum AST	53
5.4	Darstellung rekursiver Listenstrukturen	53
5.5	Typisierung und interne AST-Repräsentation	58
5.6	Interpreterarchitektur im Vergleich	59

5.7	Auswertung arithmetischer Ausdrücke	59
5.8	Auswertung boolescher Bedingungen	63
5.9	Umgebungsmodell und Programmausführung	63
5.10	Ergebnisse der Case Study	65
6	Fazit	66
7	Literaturverzeichnis	68
A	Anhang	69
A.1	Verzeichnis der portierten Lehrnotebooks	69

Abbildungsverzeichnis

Tabellenverzeichnis

1	Übersicht der übersetzten Notebooks im Repository	69
---	---	----

List of Listings

1	Mengen in Python: Das erwartete Verhalten	2
2	Mengen in TypeScript: Das fehlerhafte Verhalten	2
3	Unterschiedliche Operationen erfordern unterschiedliche Typen . .	5
4	Dynamische Typisierung in Python	6
5	Dynamische Typisierung in JavaScript	6
6	Typfehler wird erst zur Laufzeit sichtbar	6
7	Eine untypisierte Funktion in JavaScript	7
8	Dieselbe Funktion in TypeScript mit expliziten Typen	7
9	Naive Übersetzung einer Funktion nach TypeScript	8
10	Typprüfung zur Laufzeit in Python	8
11	Typprüfung zur Laufzeit in JavaScript und TypeScript	8
12	Der Typ <code>any</code> hebt die Typprüfung aus	9
13	Explizite Umwandlung in Python	9
14	Implizite und explizite Umwandlung in JavaScript	10
15	Eine Type Assertion ist keine echte Umwandlung	10
16	Unpräzise vs. präzise formulierte Funktion in TypeScript	11
17	Atomare und zusammengesetzte TypeScript-Typen	12
18	Union Type aus Literal Types in TypeScript	13
19	Literal Types in Python	13
20	Ein möglicher fehlender Rückgabewert in TypeScript	13
21	Möglicher fehlender Rückgabewert in Python	14
22	Fehlerhafte Nutzung ohne vorherige Prüfung	14
23	Explizite Behandlung eines möglichen <code>undefined</code> -Werts	15
24	Der Typ <code>unknown</code> erzwingt eine vorherige Prüfung	15
25	Einengung von <code>unknown</code> durch Typprüfung im Kontrollfluss	16
26	Type Hints in Python	16
27	Python <code>Any</code> vs. TypeScript <code>unknown</code>	17
28	Ein Interface als Vertrag über benötigte Methoden	17
29	Eine Klasse erfüllt ein Interface über ihre reine Struktur	18
30	Ein Protocol in Python als statischer Vertrag	18
31	Feste Datenstruktur mit <code>TypedDict</code> in Python	19
32	Objektstrukturen und Aliase: <code>interface</code> und <code>type</code> in TypeScript . .	20
33	Eine generische Funktion in TypeScript	20
34	Eine generische Funktion in Python	21
35	Generics mit strukturellen Schranken im Vergleich	21
36	Type Narrowing durch Kontrollfluss und <code>typeof</code>	22
37	Klassenprüfung mit <code>instanceof</code>	22

38	Vollständige Fallunterscheidung mit <code>never</code>	23
39	Mengenverhalten in Python	25
40	Mengenverhalten in TypeScript	26
41	Hinzufügen von Elementen in Python und mit <code>RecursiveSet</code>	27
42	Mitgliedschaftstest in Python und mit <code>RecursiveSet</code>	27
43	Entfernen von Elementen in Python und mit <code>RecursiveSet</code>	28
44	Roboter-Koordinaten: Tupel in Python vs. TypeScript	29
45	Dictionaries in Python vs. <code>RecursiveMap</code> in TypeScript	30
46	Der Freeze-Contract bei geschachtelten Mengen	31
47	Arbeiten mit einer veränderlichen Kopie	32
48	Vereinigung zweier Mengen in Python und mit <code>RecursiveSet</code>	33
49	Schnittmenge zweier Mengen in Python und mit <code>RecursiveSet</code>	33
50	Differenz zweier Mengen in Python und mit <code>RecursiveSet</code>	34
51	Symmetrische Differenz in Python und mit <code>RecursiveSet</code>	34
52	Python: Mengenkompensation mit Selektion und Transformation	35
53	TypeScript: Funktionale Pipeline mit <code>filterMap</code> und Lambdas	36
54	Python: Verschachtelte Mengenkompensation	37
55	TypeScript: Direkte Generierung und Vereinigung durch <code>flatMap</code>	37
56	Ply-Grammatik in Python	40
57	Lezer-Grammatik in TypeScript	40
58	Umsetzung Lezer-Grammatik in TypeScript	43
59	Erstellung des Parsers in TypeScript	44
60	<code>ParseStmt</code> Funktion in TypeScript	46
61	Ply-Grammatik	50
62	Lezer-Grammatik-Teil1	51
63	Lezer-Grammatik-Teil2	52
64	Direkte AST-Konstruktion in PLY Teil1	54
65	Direkte AST-Konstruktion in PLY Teil2	55
66	Direkte AST-Konstruktion in PLY Teil3	56
67	CST-zu-AST-Transformation in TypeScript	57
68	Rekursive Listen in der Python-Version	57
69	Verarbeitung rekursiver Listen in TypeScript	58
70	AST-Typdefinition in TypeScript	58
71	Ausführung von Anweisungen in Python	59
72	Ausführung von Anweisungen in TypeScript	60
73	Arithmetische Auswertung in Python	61
74	Arithmetische Auswertung in TypeScript Teil 1	62
75	Boolesche Auswertung in Python	63
76	Boolesche Auswertung in TypeScript	64

77	Programmausführung in Python	64
78	Programmausführung in TypeScript	65

1 Einleitung

Die Theoretische Informatik bildet ein wichtiges Fundament im Informatikstudium. Themen wie Automaten oder formale Sprachen können anfangs jedoch sehr abstrakt wirken. Um diese theoretischen Konzepte greifbarer zu machen, kommen in der Vorlesung „Theoretische Informatik III“ interaktive Notebooks zum Einsatz. Dort wird erklärender Text direkt mit ausführbarem Programmcode gemischt. Bislang basieren diese Notebooks auf der Programmiersprache Python. Studierende können so theoretische Algorithmen direkt ausprobieren, verändern und live nachvollziehen.

Diese Arbeit befasst sich mit der Übertragung dieser Lehrinhalte in die Programmiersprache TypeScript. Der Fokus liegt dabei auf einer Portierung, welche nicht nur die Syntax übersetzt, sondern die Konzepte der formalen Sprachen durch feste, verbindliche Datenstrukturen explizit modelliert.

1.1 Problemstellung

Ausgangspunkt dieser Arbeit sind interaktive Python-Notebooks, die Konzepte der theoretischen Informatik demonstrieren. Werden diese Lehrmaterialien in das TypeScript-Ökosystem übertragen, reicht es jedoch nicht aus, lediglich die Syntax der Sprache zu übersetzen. Es entstehen fundamentale technische Hürden, da beide Sprachen Daten auf unterschiedliche Weise verarbeiten. Folgende zwei Problemfelder stehen dabei im Fokus:

Zu späte Fehlererkennung bei verschachtelten Daten

Python ist eine dynamische Sprache. Das bedeutet, dass Variablen keine festen Typen haben und Fehler oft erst dann auffallen, wenn das Programm ausgeführt wird und abstürzt. Zwar gibt es Werkzeuge, um Typen in Python zu prüfen, in der interaktiven Umgebung eines Jupyter-Notebooks übersehen diese jedoch häufig Fehler.

In der Vorlesung arbeiten wir ständig mit tief verschachtelten Objekten – beispielsweise Datenstrukturen, bei denen eine Liste wieder viele weitere Objekte und Listen enthält. Fehlt einem dieser verschachtelten Elemente versehentlich eine wichtige Eigenschaft, merkt Python das erst, wenn der Code im Hintergrund genau dieses fehlerhafte Element aufruft. Wir benötigen stattdessen eine Sprache, die uns schon während des Tippens garantiert, dass unsere Datenstrukturen über alle Ebenen hinweg fehlerfrei und vollständig aufgebaut sind.

Das unerwartete Verhalten von Mengen (Sets)

Das gravierendste Hindernis bei der Übersetzung zeigt sich jedoch bei der Arbeit mit Mengen (Sets). In der theoretischen Informatik ist es unumgänglich, berechnete Zustände in Mengen zu speichern, um Duplikate zu vermeiden oder später zu prüfen, ob ein Zustand bereits erreicht wurde.

In Python funktioniert das exakt so, wie man es intuitiv erwartet. Fügen wir ein Tupel in eine Menge ein und fragen danach ab, ob ein Tupel mit denselben Zahlen enthalten ist, liefert Python das erwartete Ergebnis:

```
1  menge = set()
2  menge.add((1, 2))
3
4  # Die Abfrage liefert das erwartete Ergebnis
5  print((1, 2) in menge) # Ausgabe: True
```

Listing 1: Mengen in Python: Das erwartete Verhalten

Ein naiver Übersetzungsversuch nach TypeScript führt hier jedoch zu einem fundamentalen Bruch in der Logik. Nutzen wir die Standard-Datenstruktur Set in TypeScript auf die exakt gleiche Weise, erhalten wir plötzlich ein falsches Ergebnis:

```
1  const menge = new Set();
2  menge.add([1, 2]);
3
4  // TypeScript findet das Array überraschenderweise nicht
5  console.log(menge.has([1, 2])); // Ausgabe: false
```

Listing 2: Mengen in TypeScript: Das fehlerhafte Verhalten

Obwohl das Array `[1, 2]` im Code zuvor offensichtlich in die Menge eingefügt wurde, behauptet TypeScript, es sei nicht vorhanden. Dieser zunächst unerklärliche Unterschied führt bei einer direkten Portierung dazu, dass Logik-Abfragen fehlschlagen, Endlosschleifen entstehen und Algorithmen falsche Ergebnisse liefern. Die Ursachenforschung und Behebung dieses Verhaltens ist die zentrale technische Herausforderung dieser Arbeit.

1.2 Zielsetzung

Das übergeordnete Ziel dieser Arbeit ist es, die Lehrinhalte der Vorlesung „Theoretische Informatik III“ vollständig nach TypeScript zu übersetzen. Die neue Co-debasis soll die bisherigen Python-Notebooks ersetzen und dabei die klaren Strukturvorgaben der neuen Sprache nutzen, um Fehler frühzeitig zu vermeiden.

Daraus leiten sich folgende konkrete Teilziele ab:

1. **Strukturierte Neuimplementierung:** Die Algorithmen der Vorlesung sollen in TypeScript portiert werden. Die dynamische, oft fehleranfälligere Natur von Python soll dabei durch verbindliche Datenstrukturen und Typvorgaben ersetzt werden.
2. **Lösung des Mengen-Problems:** Um die Algorithmen fehlerfrei auszuführen, muss das unerwartete Verhalten von TypeScripts Set analysiert und behoben werden. Ziel ist die Entwicklung einer maßgeschneiderten Datenstruktur, die sich exakt wie ein Python-Set verhält und verschachtelte Objekte verlässlich erkennt.
3. **Einheitliche Ausführungsumgebung:** Da die Notebooks lokal ausgeführt werden, muss eine standardisierte Umgebung geschaffen werden, die unabhängig vom Betriebssystem identisch funktioniert. Ein wichtiges technisches Teilziel ist es dabei, die Hintergrundprozesse der Notebooks so anzupassen, dass TypeScripts strenge Prüfregele auch bei der interaktiven, schrittweisen Ausführung zuverlässig erzwungen werden. Der Einrichtungsaufwand für Studierende soll durch eine klare Schritt-für-Schritt-Anleitung minimal gehalten werden.

1.3 Methodik und Abgrenzung

Die methodische Durchführung dieser Arbeit folgt einem fokussierten Ansatz. Der Umfang der Portierung beschränkt sich auf die nummerierten Lehrnotebooks der Vorlesung sowie deren unmittelbare technische Abhängigkeiten. Übungsnotebooks, die primär dem unbetreuten Selbststudium dienen, sind explizit vom Umfang dieser Arbeit ausgeklammert. Die genaue Auswahl der zu übersetzenden Artefakte erfolgte in Abstimmung mit dem Betreuer und ist im Anhang (siehe Abschnitt A.1) tabellarisch aufgeführt.

Ein zentraler methodischer Aspekt betrifft die didaktische Aufbereitung: Die begleitenden Erklärtexpte der Notebooks wurden nicht isoliert übersetzt, sondern inhaltlich an die neuen Strukturen von TypeScript angepasst. Dabei wurde bewusst darauf verzichtet, die neuen Konzepte kontinuierlich mit der alten Python-Implementierung zu vergleichen. Die Notebooks sollen als eigenständiges Lehrmaterial funktionieren, das die theoretischen Konzepte nativ und in sich geschlossen vermittelt.

Auf technischer Ebene war die Minimierung von Systemabhängigkeiten ein wesentliches Entwurfsziel. Insbesondere für die grafische Visualisierung von Automaten und Graphen wurde darauf geachtet, keine komplexen lokalen Installationen

von externer Software vorauszusetzen. Stattdessen kommen integrierbare, plattformunabhängige Lösungen zum Einsatz, um die Ausführung auf verschiedenen Betriebssystemen zu garantieren und den Einstieg für die Studierenden so barrierefrei wie möglich zu gestalten.

Zur Gewährleistung der Reproduzierbarkeit und für die freie Zugänglichkeit der Ergebnisse wird der gesamte entwickelte Quellcode in einem öffentlichen Repository bereitgestellt. Die Implementierung ist unter der URL <https://github.com/cstrerath/formal-languages-typescript> im Verzeichnis TypeScript abrufbar.

2 Theoretische Grundlagen der Typisierung

Beim Übergang von den interaktiven Python-Notebooks der Vorlesung zu einer TypeScript-Implementierung ändert sich nicht nur die Syntax. Der fundamentalste Unterschied liegt im Umgang mit Datentypen. Die in Python erprobten Algorithmen wurden ursprünglich in einer hochdynamischen Umgebung entwickelt und mittels „mypy“ typisiert. TypeScript verwendet einen anderen Ansatz, die Typisierung ist der zentrale Aspekt der Sprache, daher ist ebenjene in diesem Projekt kein Randthema, sondern das tragende Fundament der gesamten Codebasis.

2.1 Semantik und Funktion von Datentypen

Im Arbeitsspeicher eines Computers bestehen alle Daten letztlich nur aus Bitfolgen, also aus Nullen und Einsen. Für den Prozessor ist eine solche Bitfolge zunächst bedeutungslos. Erst durch einen „Typ“ erhält sie eine fachliche Interpretation. Der Typ legt fest, ob ein Wert beispielsweise als Zahl, als Text, als Wahrheitswert oder als komplexe Datenstruktur behandelt wird.

Man kann einen Typ daher als eine Art Vertrag verstehen. Dieser Vertrag beschreibt die Form eines Wertes und definiert, welche Operationen auf ihm sinnvoll und erlaubt sind. Ein kurzes Python-Beispiel (siehe Listing 3) verdeutlicht dieses Prinzip: Eine Zahl mit einer anderen Zahl zu summieren ist logisch, ebenso die Verkettung zweier Strings. Die Addition von String und Zahl führt hingegen zu einem inhaltlichen Konflikt.

```
1 zahl = 42
2 text = "42"
3 print(zahl + 1)      # sinnvoll: 43
4 print(text + "!")    # sinnvoll: "42!"
5 print(text + 1)      # nicht sinnvoll: Text und Zahl passen hier nicht zusammen
```

Listing 3: Unterschiedliche Operationen erfordern unterschiedliche Typen

Ob eine Operation zulässig ist, hängt demnach maßgeblich vom Typ ab. Genau hier setzen Typsysteme an: Sie strukturieren die Bedeutung von Daten und dienen als essenzieller Sicherheitsmechanismus. Wenn ein Programm versucht, eine Operation auf einem ungeeigneten Wert auszuführen, liegt oft kein syntaktischer Fehler vor, sondern ein inhaltlicher Widerspruch. Ein Typsystem macht solche Widersprüche sichtbar, bevor sie zu schwer nachvollziehbaren Abstürzen im laufenden Programm führen.

Prinzipien der dynamischen Typisierung

Python und JavaScript sind dynamisch typisierte Sprachen. Das bedeutet, dass Variablen nicht dauerhaft an einen vorher festgelegten Typ gebunden sind. Der Typ ergibt sich stattdessen aus dem Wert, der zur Laufzeit gerade in der Variablen gespeichert ist. Wie Listing 4 zeigt, kann eine Variable in Python nahtlos nacheinander völlig unterschiedliche Datentypen aufnehmen.

```
1 value = 42
2 value = "Hallo"
3 value = [1, 2, 3]
4 print(value)
```

Listing 4: Dynamische Typisierung in Python

Dasselbe Grundprinzip findet sich unverändert in JavaScript wieder (siehe Listing 5), wenn man das Keyword `let` verwendet.

```
1 let value = 42;
2 value = "Hallo";
3 value = [1, 2, 3];
4 console.log(value);
```

Listing 5: Dynamische Typisierung in JavaScript

Diese Flexibilität erleichtert das schnelle Schreiben von Programmcode. Gleichzeitig verschiebt sie einen großen Teil der Verantwortung auf die Entwickelnden, da sich oft erst im Moment der Ausführung zeigt, ob eine Operation zum aktuellen Wert passt. Die möglichen Folgen dieser späten Auswertung demonstriert Listing 6.

```
1 value = [1, 2, 3]
2 print(value + 1)
```

Listing 6: Typfehler wird erst zur Laufzeit sichtbar

Obwohl der inhaltliche Fehler, die Addition von Liste und Ganzzahl, im Quelltext sofort ersichtlich ist, bemerkt Python ihn erst bei der Ausführung. In komplexen Algorithmen erschwert dies die Fehlererkennung erheblich.

2.2 Statische Typisierung mittels TypeScript

TypeScript ist keine völlig eigenständige Sprache, sondern eine statische Erweiterung von JavaScript. Während JavaScript die Laufzeitgrundlage bildet, fügt

TypeScript Typinformationen hinzu, die bereits vor der Ausführung vom Compiler geprüft werden.

Zur Veranschaulichung zeigt Listing 7 zunächst eine einfache Addierfunktion in purem JavaScript.

```
1 function add(a, b) {  
2     return a + b;  
3 }  
4 console.log(add(3, 4));  
5 console.log(add("Hallo", "Welt"));
```

Listing 7: Eine untypisierte Funktion in JavaScript

Da keine Typen vorgegeben sind, akzeptiert die Funktion beliebige Eingaben. Die Flexibilität ist hoch, die eigentliche fachliche Absicht der Funktion bleibt jedoch im Dunkeln. Listing 8 zeigt im direkten Kontrast, wie TypeScript es erlaubt, diese Absicht unmissverständlich im Code festzuhalten.

```
1 function add(a: number, b: number): number {  
2     return a + b;  
3 }  
4 console.log(add(3, 4));  
5 // console.log(add("Hallo", "Welt")); // Führt zu einem Compiler-Fehler
```

Listing 8: Dieselbe Funktion in TypeScript mit expliziten Typen

Durch die expliziten Typannotationen wird die Signatur, also die formale Schnittstelle bestehend aus Funktionsnamen sowie Anzahl, Reihenfolge und Datentypen der Parameter, eindeutig. Ein fehlerhafter Aufruf mit zwei Strings widerspricht diesem Vertrag und wird vom Compiler beanstandet, lange bevor das Programm ausgeführt wird. Der hier verwendete Typ `number` ist dabei der zentrale Baustein für numerische Werte:

Definition 2.1. Der Typ `number`: Im Gegensatz zu Sprachen wie Python, C oder Java, die streng zwischen ganzen Zahlen (`int`) und Kommazahlen (`float`) unterscheiden, fasst TypeScript alle numerischen Werte unter dem Typ `number` zusammen. Programmatisch handelt es sich um 64-Bit-Fließkommazahlen nach dem IEEE-754-Standard [1]. Eine Funktion mit diesem Parameter akzeptiert folglich sowohl `add(3, 4)` als auch `add(3.14, 2.5)`.

2.2.1 Herausforderungen einer naiven Code-Migration

Eine reine Portierung von Python-Code nach TypeScript führt nicht automatisch zu echter Typsicherheit. Da TypeScript als Obermenge von JavaScript konzipiert ist, lässt sich bestehender Python-Code häufig fast unverändert (lediglich mit geänderter Syntax) übernehmen, bleibt im Kern jedoch reines JavaScript. Listing 9 demonstriert eine solche naive Übersetzung.

```
1 function formatUserId(id) {  
2     return "User_" + id;  
3 }  
4 console.log(formatUserId(5));  
5 console.log(formatUserId("admin"));  
6 console.log(formatUserId([1, 2, 3]));
```

Listing 9: Naive Übersetzung einer Funktion nach TypeScript

Die Funktion kompiliert ohne Fehlermeldung, verarbeitet aber völlig unterschiedslos Zahlen, Strings und Arrays. Die Absicht, hier die Formatierung einer numerischen Nutzer-ID, geht komplett verloren. Ein solcher Code ist zwar formal TypeScript, inhaltlich bewahrt er jedoch die Fehleranfälligkeit der dynamischen Programmierung.

Dynamische Typprüfung zur Laufzeit

Auch in untypisierten Umgebungen besitzen Daten zur Laufzeit stets einen konkreten Typ, der sich programmatisch auslesen lässt. In Python wird hierfür typischerweise die Funktion `type()` verwendet (siehe Listing 10), während JavaScript und TypeScript den Operator `typeof` nutzen (siehe Listing 11).

```
1 print(type(42))          # <class 'int'>  
2 print(type("Hallo"))    # <class 'str'>  
3 print(type([1, 2, 3]))  # <class 'list'>
```

Listing 10: Typprüfung zur Laufzeit in Python

```
1 console.log(typeof 42);    // "number"  
2 console.log(typeof "Hallo"); // "string"  
3 console.log(typeof true);  // "boolean"
```

Listing 11: Typprüfung zur Laufzeit in JavaScript und TypeScript

Implikationen fehlender Typisierung (`any`)

Werden Variablen in TypeScript nicht explizit annotiert und lässt sich ihr Typ nicht eindeutig ableiten (wie bei dem Parameter `id` in der naiven Übersetzung), greift TypeScript auf einen Fallback zurück: Es weist der Variable den Typ `any` zu. Dieser Typ nimmt eine gefährliche Sonderrolle im System ein.

Definition 2.2. Der Typ `any`: Das Schlüsselwort `any` ist der universelle Fluchtweg aus dem Typsystem. Es repräsentiert einen beliebigen JavaScript-Wert ohne jegliche Einschränkungen. Der Compiler erlaubt auf einem `any`-Wert den Aufruf beliebiger Methoden und Eigenschaften, womit die statische Typprüfung für diese Variable faktisch vollständig deaktiviert wird.

Wie Listing 12 verdeutlicht, kann `any` auch explizit vergeben werden, um den Compiler bewusst „blind“ zu machen.

```
1 let value: any = [1, 2, 3];
2 console.log(value.toUpperCase());           // stürzt zur Laufzeit ab
3 console.log(value * 2);                     // ergibt NaN
4 console.log(value.nonExistingMethod());    // stürzt zur Laufzeit ab
```

Listing 12: Der Typ `any` hebt die Typprüfung aus

Ein typsicheres Programm müsste validieren, ob `value` überhaupt die Methode `toUpperCase()` besitzt. Bei `any` entfällt diese Kontrolle, was zu Laufzeitfehlern führen kann.

Gefahren der impliziten Typkonvertierung (Koerzision)

Ein weiteres Risiko schlecht typisierten TypeScript-Codes ist die sogenannte *Koerzision* (implizite Typumwandlung). Python verlangt für fast alle Operationen explizite Umwandlungen, wie Listing 13 zeigt.

```
1 values = [1, 2, 3]
2 text = str(values)
3 print(text)           # "[1, 2, 3]"
```

Listing 13: Explizite Umwandlung in Python

JavaScript (und untypisiertes TypeScript) ist an dieser Stelle nachgiebiger und erzwingt oft stillschweigende Umwandlungen, sobald ein Operator dies nahelegt. Listing 14 stellt dieses Verhalten der expliziten Umwandlung gegenüber.

```

1 const values = [1, 2, 3];
2 const implicitText = values + "";    // Koerzision
3 const explicitText = String(values); // Explizite Umwandlung
4 console.log(implicitText);           // "1,2,3"

```

Listing 14: Implizite und explizite Umwandlung in JavaScript

Type Assertions als trügerische Compiler-Anweisung

Neben `any` bietet TypeScript mit der Typbehauptung einen weiteren Mechanismus, der das Typsystem aufweicht und für Einsteiger oft wie eine bequeme Typumwandlung (Type Cast) wirkt. Dies ist jedoch ein Trugschluss.

Definition 2.3. Die Type Assertion (`as`): Das Schlüsselwort `as` ist keine Operation, die zur Laufzeit ausgeführt wird. Es ist eine reine Compiler-Direktive. Sie zwingt den TypeScript-Compiler dazu, einen Wert ab sofort als den angegebenen Typ zu behandeln, lässt den Wert im Arbeitsspeicher aber völlig unangetastet.

Listing 15 verdeutlicht diese trügerische Sicherheit.

```

1 let values: any = [1, 2, 3];
2
3 // Keine echte Laufzeit-Umwandlung, sondern nur eine Behauptung:
4 const text = values as string;
5
6 // Der Wert ist zur Laufzeit weiterhin ein Array!
7 console.log(text.toUpperCase()); // Laufzeitfehler

```

Listing 15: Eine Type Assertion ist keine echte Umwandlung

Das inhaltliche Problem wurde somit nicht gelöst, sondern lediglich vor der Typprüfung versteckt, zur Laufzeit ist die Methode auf dem Array nicht definiert und es kommt zum Absturz.

Methodischer Verzicht auf Typ-Bypässe

Weder `any` noch `as` sind neutrale Komfortfunktionen. Es sind Mechanismen, die die Wirkung des Typsystems gezielt aushebeln. Das erklärte Ziel dieser Studienarbeit war es, den Code nach TypeScript, und nicht bloß nach JavaScript in `.ts`-Dateien, zu überführen. Folglich wurde architektonisch in der gesamten Implementierung auf den Einsatz von `any` und ungesicherten `as`-Behauptungen verzichtet.

2.2.2 Explizite Typannotationen

Der professionelle Gegenentwurf zu Ausweichmechanismen ist die explizite Typannotation. Dabei wird im Programmtext unmissverständlich angegeben, welcher Typ für Variablen, Parameter oder Rückgabewerte zwingend erwartet wird.

Listing 16 zeigt den Unterschied anhand einer Funktion, die aus einer numerischen ID einen formatierten Text generiert.

```
1 // Unpräzise: Lässt völlig unklar, was id sein soll
2 function formatUserIdLoose(id) {
3     return "User_" + id;
4 }
5
6 // Präzise: Zwingender Vertrag durch explizite Annotationen
7 function formatUserIdStrict(id: number): string {
8     return "User_" + id;
9 }
```

Listing 16: Unpräzise vs. präzise formulierte Funktion in TypeScript

Die strikte Variante formuliert einen eindeutigen Vertrag, der keinen Interpretationsspielraum zulässt. Die Signatur verlangt nicht nur eine `number` als Eingabe, sondern garantiert auch einen spezifischen Rückgabebetyp, in diesem Fall einen Text:

Definition 2.4. Der Typ `string`: Dieser primitive Typ repräsentiert jegliche Form von Textdaten. Er besteht aus einer unveränderlichen (immutablen) Sequenz von 16-Bit-Werten nach dem UTF-16-Standard. Einmal im Speicher angelegt, kann ein String nicht modifiziert werden; Operationen wie String-Verkettungen erzeugen stets eine neue Zeichenkette.

Atomare und strukturierte Basistypen

Neben den bereits eingeführten primitiven Werten wie Zahlen und Texten, ist ein dritter atomarer Baustein essenziell für die Steuerung des Programmflusses:

Definition 2.5. Der Typ `boolean`: Er repräsentiert logische Wahrheitswerte und kann ausschließlich die Zustände `true` oder `false` annehmen. Er entspricht exakt dem `bool`-Typ aus Python und ist essenziell für Kontrollfluss-Bedingungen.

Zusätzlich zu diesen atomaren Bausteinen erlaubt TypeScript die Definition zusammengesetzter Strukturen. Wie Listing 17 demonstriert, können Arrays oder spezifisch geformte Objekte direkt annotiert werden.

```

1 // Atomare Typen
2 let username: string = "alice";
3 let loginCount: number = 5;
4 let isActive: boolean = true;
5
6 // Zusammengesetzte Typen
7 let roles: string[] = ["admin", "staff"];
8 let userProfile: { id: number; name: string } = {
9     id: 42,
10    name: "Alice"
11 };

```

Listing 17: Atomare und zusammengesetzte TypeScript-Typen

Bereits die simple Typdefinition von `userProfile` legt eine klare Struktur mit zwei benannten und typisierten Bestandteilen fest. Die Fähigkeit, komplexe Verträge im Code zu verankern, ist Grundlage sicherer Architekturen.

Eingrenzung von Wertebereichen durch Literal und Union Types

Oft reicht es nicht aus, einen Wert pauschal als `string` zu deklarieren. Wenn fachlich nur eine feste Menge spezifischer Ausprägungen zulässig ist (etwa bestimmte Nutzerrollen), bietet TypeScript zwei eng verwobene Konzepte:

Definition 2.6. Literal Types: Ein Literal Type beschreibt nicht eine ganze Menge von Werten, sondern repräsentiert exakt einen einzigen, konkreten Wert (z. B. exakt die Zeichenkette `"admin"`).

Definition 2.7. Union Types (I): Ein Union Type kombiniert mehrere Typen durch ein logisches „Oder“. Eine Variable dieses Typs darf zur Laufzeit jeden der verknüpften Typen annehmen.

Ihre immense Flexibilität gewinnen Literal Types gerade durch diese Kombination mit Union Types. Listing 18 zeigt, wie sich Zustandsräume dadurch präzise eingrenzen lassen.

Dieses Konzept transformiert lose Text-Konventionen in maschinenprüfbare Verträge. Python unterstützt dies ebenfalls sehr ähnlich und nutzt dafür `Literal` aus dem `typing`-Modul, wie Listing 19 belegt.

Modellierung fehlender Werte (`null` und `undefined`)

In realen Programmen tritt häufig der Fall auf, dass ein gesuchter Wert schlicht nicht vorhanden ist. Ein präzises Typsystem muss die Möglichkeit eines solchen fehlenden Wertes ausdrücklich modellieren, anstatt einfach darauf zu hoffen,


```

1 type Role = "admin" | "user" | "guest";
2
3 let currentRole: Role = "user";
4 currentRole = "admin";
5 // currentRole = "superuser";
6 // Compiler-Fehler: Typ '"superuser"' ist nicht zuweisbar

```

Listing 18: Union Type aus Literal Types in TypeScript

```

1 from typing import Literal
2
3 Role = Literal["admin", "user", "guest"]
4
5 def is_privileged(role: Role) -> bool:
6     return role == "admin"

```

Listing 19: Literal Types in Python

dass der aufrufende Code damit zurechtkommt. TypeScript verwendet dafür zwei spezifische Typen, die verschiedene Nuancen der Abwesenheit ausdrücken:

Definition 2.8. Die Typen `undefined` und `null`: Der Typ `undefined` bedeutet typischerweise, dass ein Wert noch nicht initialisiert wurde oder in einem Suchvorgang nicht gefunden werden konnte. Der Typ `null` wird hingegen meist genutzt, um die absichtliche, bewusste Abwesenheit eines Wertes durch den Entwickler zu markieren. Beide Typen repräsentieren zur Laufzeit einen fehlenden Wert, haben aber eine leicht unterschiedliche Semantik.

Die Suche nach einer E-Mail-Adresse in einer Datenbank, wie in Listing 20 dargestellt, verdeutlicht dieses Prinzip.

```

1 function findEmail(username: string): string | undefined {
2     if (username === "admin") {
3         return "admin@example.com";
4     }
5     return undefined;
6 }

```

Listing 20: Ein möglicher fehlender Rückgabewert in TypeScript

Da die Funktion nicht zwingend ein Ergebnis findet, reicht der Rückgabebetyp `string` allein nicht aus. Die Signatur `string | undefined` nutzt einen Union Type und

hält somit ausdrücklich fest, dass entweder ein String oder eben kein nutzbarer Wert zurückkommt.

Python modelliert dieses Problem auf der Ebene der Laufzeitwerte ganz ähnlich, wie Listing 21 zeigt. Dort wird ein potenziell fehlender Wert typischerweise durch `None` ausgedrückt.

```
1 def find_email(username):  
2     if username == "admin":  
3         return "admin@example.com"  
4     return None
```

Listing 21: Möglicher fehlender Rückgabewert in Python

Der Grundgedanke ist in beiden Sprachen identisch: Ein Rückgabewert kann vorhanden sein oder fehlen. Entscheidend ist jedoch, wie streng diese Möglichkeit anschließend durch den Compiler der jeweiligen Sprache eingefordert wird.

Strikte Nullwert-Prüfung (`strictNullChecks`)

Die bloße Existenz von `null` und `undefined` erzeugt noch keine automatische Typsicherheit. Ob der Compiler diese Fälle als eigenständige Problemquellen ernst nimmt, steuert in TypeScript eine spezifische Konfigurationseinstellung:

Definition 2.9. Die Compiler-Option `strictNullChecks`: Ist diese Option deaktiviert, behandelt TypeScript fehlende Werte nachsichtig, `null` und `undefined` dürfen überall dort übergeben werden, wo eigentlich beispielsweise ein `string` erwartet wird. Ist sie jedoch aktiviert, trennt der Compiler diese Typen strikt von regulären Werten. Ein potenziell fehlender Wert muss dann zwingend im Code auf seine Existenz geprüft werden, bevor auf ihn zugegriffen werden darf.

Führt man das vorherige E-Mail-Beispiel fort, wird die Tragweite dieser Option in Listing 22 sofort sichtbar.

```
1 const email = findEmail("guest");  
2 console.log(email.toUpperCase()); // Compiler-Fehler bei aktivem strictNullChecks
```

Listing 22: Fehlerhafte Nutzung ohne vorherige Prüfung

Diese Zeile ist hochgradig fehleranfällig. Liefert die Suche `undefined`, schlägt der Aufruf von `toUpperCase()` zur Laufzeit fehl. Mit aktiviertem `strictNullChecks` macht der TypeScript-Compiler diesen Widerspruch schon während des Tippens im Editor sichtbar und verweigert die Übersetzung.

Die saubere Lösung besteht darin, den fehlenden Wert im Kontrollfluss explizit zu behandeln, wie Listing 23 demonstriert.

```
1 const email = findEmail("guest");
2 if (email !== undefined) {
3     console.log(email.toUpperCase()); // email kann nur string sein
4 }
```

Listing 23: Explizite Behandlung eines möglichen undefined-Werts

Einer der häufigsten Fehler in Programmen besteht darin, blind davon auszugehen, dass ein Wert vorhanden ist. Für die Notebooks war `strictNullChecks` deshalb keine kosmetische Zusatzregel, sondern eine elementare Sicherheitsmaßnahme, welche über einen Patch am Notebookkernel eingebaut wurde.

Der Typ `unknown` als sichere Abstraktion

Neben `null` und `undefined` bietet TypeScript einen weiteren wichtigen Spezialfall an, um mit Unsicherheit umzugehen. Dieser ist besonders dann nützlich, wenn zwar feststeht, dass ein Wert existiert, dessen genauer Typ aber noch nicht bekannt ist (beispielsweise bei externen Eingaben).

Definition 2.10. Der Typ `unknown`: Dieser Typ ist das typsichere Gegenstück zu `any`. Während `any` die Typprüfung faktisch deaktiviert, verbietet `unknown` zunächst jegliche Interaktion. Der Compiler zwingt den Entwickler dazu, den Wert durch Laufzeitprüfungen präzise einzuengen, bevor typspezifische Methoden aufgerufen werden dürfen.

Listing 24 zeigt, wie der Compiler unsichere Operationen auf einem `unknown`-Wert rigoros blockiert.

```
1 let value: unknown = [1, 2, 3];
2
3 // Nicht erlaubt, solange der Typ noch unbekannt ist:
4 // console.log(value.toUpperCase()); // Compiler-Fehler
```

Listing 24: Der Typ `unknown` erzwingt eine vorherige Prüfung

Erst nach einer erfolgreichen Typprüfung im Kontrollfluss schaltet der Compiler die typspezifischen Methoden frei, was in Listing 25 ersichtlich wird.

Auf diese Weise lassen sich unsichere Daten sicher modellieren, ohne den Type Checker auszuhebeln.

```
1 let value: unknown = "Alice";
2
3 if (typeof value === "string") {
4     // Innerhalb dieses Blocks weiß der Compiler, dass value ein string ist.
5     console.log(value.toUpperCase());
6 }
```

Listing 25: Einengung von `unknown` durch Typprüfung im Kontrollfluss

Vergleich mit statischer Analyse in Python (mypy)

Auf den ersten Blick besitzen Python und TypeScript ähnliche Ausdrucksmittel für Typen. Wie Listing 26 zeigt, lassen sich auch in Python Variablen, Parameter und Rückgabewerte mittels Type Hints annotieren.

```
1 def format_user_id(user_id: int) -> str:
2     return f"User_{user_id}"
```

Listing 26: Type Hints in Python

Ein fundamentaler Unterschied besteht jedoch in der Durchsetzung: Python ignoriert diese Type Hints zur Laufzeit vollständig. Ein fehlerhafter Aufruf mit falschen Datentypen führt nicht zwingend zu einem sofortigen Fehler. Um die Typinformationen systematisch zu prüfen, wird ein externes Werkzeug wie *mypy* benötigt, das den Code statisch analysiert.

Zudem fehlt Python ein strenges Äquivalent zum sicheren `unknown`. Der Typ `Any` in Python entspricht vielmehr dem unsicheren `any` in TypeScript. Listing 27 stellt diese Konzepte direkt gegenüber.

2.2.3 Strukturelles Typing und Interface-Verträge

Neben primitiven Werten erfordert die Modellierung von Software verlässliche Verträge zwischen Komponenten. Ein solcher Vertrag legt fest, welche Eigenschaften oder Methoden ein Wert besitzen muss, um genutzt werden zu können. TypeScript verwendet dafür unter anderem *Interfaces*.

Entscheidend ist dabei ein Konzept, das TypeScript grundlegend von Sprachen wie Java oder C# unterscheidet:

```

1 from typing import Any
2
3 def normalize(value: Any) -> str:
4     return str(value) # mypy erlaubt hier völlig beliebige Eingaben

```

```

1 function normalize(value: unknown): string {
2     if (typeof value === "string") {
3         return value;
4     }
5     return String(value); // TypeScript erzwingt die Prüfung oder Konvertierung
6 }

```

Listing 27: Python Any vs. TypeScript unknown

Definition 2.11. Strukturelles Typing (Statisches Duck-Typing): TypeScript prüft Typkompatibilität nicht anhand einer expliziten Klassenhierarchie (Nominales Typing), sondern rein anhand der „Gestalt“ (Struktur) eines Objekts. Besitzt ein Objekt alle Eigenschaften und Methoden, die ein Interface verlangt, gilt es als kompatibel zu diesem Interface, unabhängig davon, ob es explizit davon erbt oder nicht.

Listing 28 definiert einen solchen strukturellen Vertrag in Form des Interfaces `Displayable`.

```

1 interface Displayable {
2     getLabel(): string;
3     isActive(): boolean;
4 }

```

Listing 28: Ein Interface als Vertrag über benötigte Methoden

Dieses Interface fordert lediglich das Vorhandensein zweier spezifischer Methoden. Listing 29 demonstriert, dass jede Klasse, die diese Signatur zufällig oder absichtlich erfüllt, automatisch typkompatibel ist.

Dieser strukturelle Ansatz entkoppelt Funktionalität von starren Klassenhierarchien. Datenstrukturen können polymorph mit völlig unterschiedlichen Elementen arbeiten, solange diese den vertraglich zugesicherten Methodenapparat bereitstellen.

```

1 class UserAccount {
2     constructor(public username: string, public active: boolean) {}
3
4     getLabel(): string {
5         return this.username;
6     }
7
8     isActive(): boolean {
9         return this.active;
10    }
11 }
12
13 // Zuweisung ist zulässig, da Struktur des Objekts mit Interface übereinstimmt:
14 const item: Displayable = new UserAccount("alice", true);

```

Listing 29: Eine Klasse erfüllt ein Interface über ihre reine Struktur

Äquivalente Strukturierung in Python (Protocol und TypedDict)

Python unterstützt strukturelles Denken zur Laufzeit nativ durch sein dynamisches Duck-Typing. Um dieses Verhalten jedoch statisch überprüfbar zu machen, bietet das `typing`-Modul (in Kombination mit `mypy`) spezifische Hilfsmittel an.

Analog zu einem TypeScript-Interface ermöglicht die Klasse `Protocol` die Definition von Methoden-Verträgen. Listing 30 zeigt, wie ein solcher statischer Vertrag in Python formuliert wird.

```

1 from typing import Protocol
2
3 class Displayable(Protocol):
4     def get_label(self) -> str: ...
5     def is_active(self) -> bool: ...

```

Listing 30: Ein Protocol in Python als statischer Vertrag

Klassen müssen auch in Python nicht explizit von diesem Protocol erben, um als kompatibel zu gelten; die bloße Erfüllung der Signatur reicht aus. Während strukturelle Kompatibilität in TypeScript jedoch das fundamentale Designprinzip bildet, bleibt sie in Python ein optionales Konstrukt.

Für reine Datenobjekte (ohne eigenes Verhalten) bietet Python zusätzlich `TypedDict`. Wie Listing 31 demonstriert, lassen sich damit Dictionary-Strukturen mit festen Schlüsseln und Typen definieren.

```
1 from typing import TypedDict
2
3 class Address(TypedDict):
4     street: str
5     zip_code: int
6     city: str
```

Listing 31: Feste Datenstruktur mit TypedDict in Python

Interfaces vs. Type Aliase in TypeScript

In TypeScript gibt es für die Modellierung von Datenstrukturen gleich zwei Werkzeuge, die auf den ersten Blick identisch wirken: `interface` und `type`. Während `interface` (wie in Abschnitt 2.2.3 gezeigt) historisch aus der objektorientierten Welt stammt und primär die Gestalt von Objekten oder Klassen beschreibt, ist `type` ein deutlich universelleres Konzept:

Definition 2.12. Type Alias (`type`): Ein Type Alias gibt einem beliebigen Typ einen neuen, aussagekräftigen Namen. Im Gegensatz zu einem `interface` kann ein `type` nicht nur Objektstrukturen, sondern auch die bereits eingeführten Union Types, Tupel oder primitive Werte benennen.

Listing 32 stellt die TypeScript-Äquivalenz zum Python-`TypedDict` dar. Es zeigt, wie fließend `interface` und `type` für reine Datenobjekte ineinander übergehen, während `type` bei komplexeren Kombinationen (wie Union Types) alternativlos ist.

Für die Architektur der portierten Software bedeutet dies eine klare Arbeitsteilung: Sobald Verträge für Klassen oder polymorphes Verhalten (Methoden) definiert werden, greift die Codebasis bevorzugt auf `interface` zurück. Geht es jedoch um die Definition von reinen Datencontainern, Tupeln oder die Kombination von Typen (wie bei `UnsafeMail`), ist der `type`-Alias das Werkzeug der Wahl. Diese saubere semantische Trennung verbessert die Lesbarkeit und macht die TypeScript-Notebooks in ihrer Typbeschreibung ausdrucksstärker als die Python-Vorlage.

Generische Programmierung im Sprachvergleich

Um Algorithmen und Datenstrukturen typsicher, aber dennoch hochgradig wiederverwendbar zu gestalten, bedarf es einer weiteren Abstraktionsebene:

```

1 // Variante 1: Interface (Fokus auf die Objektform, oft nachträglich erweiterbar)
2 interface Address {
3     street: string;
4     zipCode: number;
5     city: string;
6 }
7
8 // Variante 2: Type Alias (Fokus auf einen festen Namen für eine feste Struktur)
9 type UserData = {
10     name: string;
11     email: string;
12     address: Address; // Nahtlose Kombination von Type und Interface
13 };
14
15 // Die exklusive Stärke von 'type': Benennung von Unions oder Tupeln
16 // (Dies wäre mit einem 'interface' syntaktisch nicht möglich)
17 type UnsafeMail = string | undefined;

```

Listing 32: Objektstrukturen und Aliase: interface und type in TypeScript

Definition 2.13. Generische Programmierung (Generics): Generics fungieren als Variablen für Typen. Anstatt eine Funktion starr auf einen Typen festzulegen oder auf das unsichere `any` auszuweichen, wird ein Typ-Parameter (meist `T`) definiert. Dieser Platzhalter wird erst zum Zeitpunkt des tatsächlichen Aufrufs durch den konkreten Datentyp ersetzt.

Listing 33 demonstriert den Einsatz einer solchen generischen Funktion in TypeScript.

```

1 function first<T>(items: T[]): T | undefined {
2     if (items.length === 0) return undefined;
3     return items[0];
4 }
5
6 // firstNumber ist number | undefined
7 const firstNumber = first<number>([10, 20, 30]);
8 // firstName ist string | undefined
9 const firstName = first<string>(["Alice", "Bob"]);

```

Listing 33: Eine generische Funktion in TypeScript

Dasselbe Konzept wird in Python mittels `TypeVar` umgesetzt, wie der direkte Vergleich in Listing 34 zeigt.

Die wahre architektonische Stärke von Generics entfaltet sich in der Kombi-


```

1 from typing import TypeVar
2
3 T = TypeVar("T")
4
5 def first(items: list[T]) -> T | None:
6     if len(items) == 0:
7         return None
8     return items[0]

```

Listing 34: Eine generische Funktion in Python

nation mit strukturellen Schranken (Type Constraints). Listing 35 veranschaulicht, wie sich mit dem Schlüsselwort `extends` (in TypeScript) oder dem Parameter `bound` (in Python) erzwingen lässt, dass der variabel eingesetzte Typ einen bestimmten Basis-Vertrag erfüllen muss.

```

1 // TypeScript: T muss mindestens die Methoden aus 'Displayable' besitzen
2 function filterActive<T extends Displayable>(items: T[]): T[] {
3     return items.filter(item => item.isActive());
4 }

```

```

1 # Python: S wird an das Protocol 'Displayable' gebunden
2 S = TypeVar("S", bound=Displayable)
3
4 def filter_active(items: list[S]) -> list[S]:
5     return [item for item in items if item.is_active()]

```

Listing 35: Generics mit strukturellen Schranken im Vergleich

Kontrollflussbasierte Typverfeinerung (Type Narrowing)

Sobald ein Wert als Union Type definiert ist, verlangt der Compiler Klarheit darüber, welcher Fall zur Laufzeit konkret vorliegt, bevor typspezifische Operationen zugelassen werden.

Definition 2.14. Type Narrowing: Dies ist der Prozess der schrittweisen Typ-Einengung. Der TypeScript-Compiler analysiert den Kontrollfluss des Codes (wie `if`-Verzweigungen) in Kombination mit Laufzeitprüfungen. Kann bewiesen werden, dass eine Variable einen bestimmten Typ hat, behandelt der Compiler sie innerhalb dieses Blocks vollautomatisch als diesen engeren Typ.

Listing 36 verdeutlicht, wie diese statische Analyse in der Praxis auf Basis des

`typeof`-Operators funktioniert.

```
1 function formatValue(value: string | number): string {
2     if (typeof value === "string") {
3         // Narrowing greift: value ist hier zwingend ein string
4         return value.toUpperCase();
5     }
6     // Durch den frühen Return oben MUSS value hier eine number sein
7     return value.toFixed(2);
8 }
```

Listing 36: Type Narrowing durch Kontrollfluss und `typeof`

Während `typeof` für primitive Datentypen geeignet ist, liefert es bei Instanzen komplexer Klassen pauschal nur `"object"`. Für die Überprüfung von Objekten bietet sich daher der Operator `instanceof` an. Listing 37 demonstriert diese klassenbasierte Verfeinerung.

```
1 class UserAccount {
2     constructor(public username: string, public active: boolean) {}
3 }
4
5 function describe(user: UserAccount | string): string {
6     if (user instanceof UserAccount) {
7         return `Konto: ${user.username} (Aktiv: ${user.active})`;
8     }
9     return `Gast: ${user}`;
10 }
```

Listing 37: Klassenprüfung mit `instanceof`

Vollständigkeitsprüfung mittels `never`-Typ

Das Type Narrowing wirft unweigerlich eine logische Folgefrage auf: Welchen Typ hat ein Wert, wenn alle theoretisch möglichen Fälle einer Union vollständig abgehandelt wurden? In TypeScript nimmt dieser logisch un erreichbare Zustand einen besonderen Typ an:

Definition 2.15. Der Typ `never`: Dieser Typ repräsentiert einen Wert, der niemals existieren darf und Code, der niemals erreicht werden sollte. Er bildet den tiefsten Punkt in der Typhierarchie. Nichts ist `never` zuweisbar.

`never` wird in der Praxis als genialer Trick für das sogenannte *Exhaustiveness Checking* (Vollständigkeitsprüfung) eingesetzt. Listing 38 zeigt, wie dieser Typ

garantiert, dass Erweiterungen des Datenmodells (z. B. ein neuer Bestellstatus im Online-Shop) vom Entwickler niemals unbemerkt übersehen werden können.

```
1 type OrderStatus = "pending" | "shipped" | "delivered";
2
3 function translateStatus(status: OrderStatus): string {
4     switch (status) {
5         case "pending": return "In Bearbeitung";
6         case "shipped": return "Versendet";
7         case "delivered": return "Zugestellt";
8
9         default: {
10             // Compile-Fehler, falls OrderStatus in Zukunft z.B. um "canceled"
11             // erweitert wird, ohne diesen Switch-Block anzupassen:
12             const exhaustiveCheck: never = status;
13             return exhaustiveCheck;
14         }
15     }
16 }
```

Listing 38: Vollständige Fallunterscheidung mit `never`

2.3 Fazit zur architektonischen Neuausrichtung

Die Herausforderung der ursprünglichen Architektur lag nicht in einer grundsätzlichen Unzulänglichkeit von *mypy* als Werkzeug. Für klassische, dateibasierte Python-Projekte ist *mypy* ein leistungsfähiger statischer Checker. Die Schwierigkeiten resultierten vielmehr aus der Diskrepanz zwischen einer nachträglich aufgesetzten statischen Typprüfung und der hochdynamischen, zellenbasierten Laufzeitumgebung von Jupyter-Notebooks.

In der Python-Umgebung musste die Typprüfung durch externe Werkzeuge wie *nb-mypy* nachgerüstet werden. Im praktischen Lehrbetrieb offenbarten sich dabei zwei gravierende architektonische Reibungspunkte:

- **Fehlendes Type Narrowing bei `match-case`:** Bei der Verwendung moderner Sprachkonstrukte wie dem strukturellen Pattern Matching (`match` und `case`) stieß die Notebook-Integration des Checkers an ihre Grenzen. Selbst wenn alle theoretisch möglichen Fälle logisch vollständig abgehandelt waren, erkannte das Tool diese Vollständigkeit (Exhaustiveness) nicht zuverlässig. Um Fehlermeldungen über angeblich fehlende Rückgabewerte zu vermeiden, mussten gezwungenermaßen toter Code geschrieben (z. B. ein redundantes `return None`) oder legitime Konstrukte per `# type: ignore` unterdrückt werden.

- **Zustandsverlust bei `%run`-Befehlen:** Um Code aus Basis-Notebooks in anderen Notebooks wiederzuverwenden, macht die interaktive Umgebung Gebrauch von Magic Commands wie `%run`. Dies korrumpierte jedoch regelmäßig den internen Typ-Zustand von *nb-mypy*. Als Workaround musste die Typprüfungs-Erweiterung manuell entladen werden, was ihren inhärenten Zweck ad absurdum führte.

Die Neuausrichtung des Projekts behält die didaktischen Vorteile der interaktiven Arbeit bei, wechselt jedoch die technologische Basis, um genau diese Brüche zu heilen. Durch den Einsatz von TypeScript in Kombination mit dem *ts/ab*-Kernel wird die Typprüfung zu einem integralen Bestandteil der Ausführungsumgebung. Der TypeScript-Kernel verifiziert den Code durch seine native Kontrollflussanalyse direkt während der Eingabe und über Zellengrenzen hinweg, ohne auf fehleranfällige Zusatzskripte oder ständige Neustarts der Prüfwerkzeuge angewiesen zu sein.

3 RecursiveSet: API und fachliche Modellierung

Wie bereits in Kapitel 1 anhand von Listing 1 und Listing 2 zu sehen war, verhalten sich Mengen in Python und TypeScript nicht gleich. Dieses Kapitel erklärt zunächst, worauf diese Unterschiede fachlich und technisch zurückzuführen sind. Anschließend wird gezeigt, wie die Bibliothek `RecursiveSet` diese Unterschiede für die Arbeit mit mathematischen Mengenstrukturen ausgleicht und dadurch eine an Python angelehnte Arbeitsweise in TypeScript ermöglicht.

3.1 Mathematische Mengen und Wohlunterscheidbarkeit

Was ist überhaupt eine Menge im mathematischen Sinn? Georg Cantor beschreibt eine Menge als „Zusammenfassung bestimmter wohlunterschiedener Objekte [...] zu einem Ganzen“ [2, S. 481]. Für die Umsetzung mengenbasierter Algorithmen in einer Programmiersprache ergibt sich daraus unmittelbar eine praktische Frage: Wann gelten zwei Elemente als gleich?

Python und TypeScript beziehungsweise JavaScript beantworten diese Frage unterschiedlich. In Python können unveränderliche, hashbare Werte wie Tupel als Elemente einer Menge verwendet werden. Zwei Tupel gelten dabei als gleich, wenn sie denselben Inhalt haben. In JavaScript werden Objekte wie Arrays in einem `Set` dagegen über ihre Referenzidentität unterschieden. Zwei Arrays mit gleichem Inhalt gelten also nur dann als gleich, wenn es sich um dieselbe Array-Instanz handelt.

Listing 39 zeigt zunächst das Verhalten in Python:

```
1 # Python: Tupel sind unveränderlich und als Set-Elemente geeignet
2 zahlen_menge: set[tuple[int, int]] = set()
3
4 zahlen_menge.add((1, 2))
5 zahlen_menge.add((2, 1))
6
7 print(zahlen_menge)           # {(1, 2), (2, 1)}
8 print((2, 1) in zahlen_menge) # True
```

Listing 39: Mengenverhalten in Python

Die Mitgliedschaftsabfrage funktioniert hier, weil Tupel unveränderlich sind und in Mengen anhand ihrer Gleichheit berücksichtigt werden können.

Anders verhält es sich in TypeScript, wie Listing 40 zeigt. Dieselbe Modellierungs-idee führt zu einem anderen Ergebnis:

```
1 type ZahlenTupel = [number, number];
2 const zahlenMenge = new Set<ZahlenTupel>();
3
4 zahlenMenge.add([1, 2]);
5 zahlenMenge.add([2, 1]);
6
7 console.log(zahlenMenge.has([2, 1])); // false
```

Listing 40: Mengenverhalten in TypeScript

Hier schlägt die Abfrage fehl, weil das neu erzeugte Array `[2, 1]` nicht dieselbe Referenz besitzt wie das zuvor eingefügte Array.

Für Algorithmen aus der diskreten Mathematik und theoretischen Informatik ist dieser Unterschied wesentlich. Python erlaubt mit unveränderlichen Werten wie Tupeln eine deutlich natürlichere Modellierung mathematischer Strukturen. In TypeScript ist für inhaltsbasierte Mengen zusammengesetzter Werte in der Regel eine zusätzliche Repräsentation oder eine eigene Datenstruktur erforderlich.

3.2 Die Datenstruktur `RecursiveSet`

Die bisherigen Beispiele haben gezeigt, dass native Mengen in TypeScript für mathematische Strukturen nur eingeschränkt geeignet sind. Insbesondere bei zusammengesetzten Werten oder geschachtelten Mengen reicht die referenzbasierte Gleichheitsprüfung der JavaScript-Laufzeit nicht aus.

Genau an dieser Stelle setzt die Bibliothek `RecursiveSet` an. Auf API-Ebene orientiert sie sich bewusst an der aus Python vertrauten Arbeit mit Mengen: Elemente können hinzugefügt, entfernt und auf Mitgliedschaft geprüft werden. Für die Benutzung bedeutet dies, dass sich `RecursiveSet` zunächst wie eine gewöhnliche Mengenstruktur anfühlt, auch wenn die Bibliothek fachlich auf strengere mathematische Modellierung ausgelegt ist.

Die grundlegenden Operationen von `RecursiveSet` sind daher bewusst nah an der aus Python bekannten Mengen-API gestaltet. Listing 41, Listing 42 und Listing 43 zeigen die drei Basisoperationen Hinzufügen, Mitgliedschaftstest und Entfernen im direkten Vergleich.

Hinzufügen von Elementen

Das Einfügen neuer Elemente erfolgt in beiden Fällen über eine explizite Mengenoperation. Listing 41 zeigt, dass sich `set` und `RecursiveSet` an dieser Stelle nahezu identisch verwenden lassen.

```
1 alphabet = {"a", "b", "c"}
2 alphabet.add("d")
3
4 print(alphabet) # {'a', 'b', 'c', 'd'}
```

```
1 import { RecursiveSet } from "recursive-set";
2
3 const alphabet = new RecursiveSet<string>("a", "b", "c");
4 alphabet.add("d");
5
6 console.log(alphabet); // {a, b, c, d}
```

Listing 41: Hinzufügen von Elementen in Python und mit RecursiveSet

Mitgliedschaftstest

Auch die Prüfung auf Mitgliedschaft gehört zu den zentralen Grundoperationen jeder Mengenstruktur. Während Python hierfür den Operator `in` verwendet, stellt `RecursiveSet`, wie viele Programmierschnittstellen in TypeScript, eine Methode `has()` bereit.

Listing 42 verdeutlicht diesen Unterschied in der Schreibweise.

```
1 alphabet = {"a", "b", "c"}
2
3 print("b" in alphabet) # True
4 print("x" in alphabet) # False
```

```
1 import { RecursiveSet } from "recursive-set";
2
3 const alphabet = new RecursiveSet<string>("a", "b", "c");
4
5 console.log(alphabet.has("b")); // true
6 console.log(alphabet.has("x")); // false
```

Listing 42: Mitgliedschaftstest in Python und mit RecursiveSet

Entfernen von Elementen

Schließlich müssen Elemente auch wieder aus einer Menge entfernt werden können. Listing 43 zeigt, dass sich auch diese Operation in beiden Sprachen sehr ähnlich ausdrücken lässt.

```

1 alphabet = {"a", "b", "c"}
2
3 alphabet.remove("b")
4 print(alphabet) # {'a', 'c'}

```

```

1 import { RecursiveSet } from "recursive-set";
2
3 const alphabet = new RecursiveSet<string>("a", "b", "c");
4
5 alphabet.remove("b");
6 console.log(alphabet); // {a, c}

```

Listing 43: Entfernen von Elementen in Python und mit RecursiveSet

Auf Ebene dieser Basisoperationen verhält sich `RecursiveSet` somit weitgehend wie eine aus Python vertraute Mengenstruktur. Der eigentliche Unterschied zeigt sich erst dann, wenn nicht nur primitive Werte, sondern auch zusammengesetzte oder rekursive Strukturen als Elemente einer Menge betrachtet werden.

3.3 Strukturierte Elemente: Die `Tuple`-Klasse

Neben Mengen stützen sich theoretische Modelle häufig auf Tupel. Ein Tupel ist eine endliche, geordnete Sequenz von Elementen. Im Unterschied zu Mengen ist die Reihenfolge der Einträge dabei wesentlich, und auch Mehrfachvorkommen sind zulässig. Das klassische Beispiel ist ein Koordinatenpaar (x, y) in einem zweidimensionalen Raum. Für zwei Punkte $P = (2, 5)$ und $Q = (5, 2)$ gilt daher mathematisch strikt $P \neq Q$.

In Python ist dieses Konzept unmittelbar in der Sprache vorhanden. Tupel sind dort unveränderliche Sequenzen und eignen sich deshalb besonders gut zur Modellierung fester, geordneter Datenstrukturen wie Koordinaten, Zustände oder Übergänge. Ein einmal definiertes Tupel kann nachträglich nicht mehr verändert werden.

Wird ein solches Modell in TypeScript mit gewöhnlichen Arrays nachgebildet, entstehen jedoch zwei grundlegende Probleme. Zum einen werden Arrays bei Vergleichen nicht über ihren mathematischen Inhalt, sondern als Objekte der Laufzeit behandelt. Zum anderen sind Arrays veränderlich, sodass einzelne Einträge nachträglich überschrieben werden können. Gerade bei Koordinaten oder Zuständen kann dies die Konsistenz einer bereits aufgebauten Struktur verletzen.

Die Bibliothek begegnet diesem Problem mit der dedizierten `Tuple`-Klasse. Sie dient als geordneter, unveränderlicher Container für strukturierte Werte und er-

gänzt `RecursiveSet` um genau die Art von Elementen, die in theoretischen Modellen regelmäßig benötigt werden. Für die Portierung der Lehrnotebooks entsteht damit ein Gegenstück zum Python-`tuple`, das sich fachlich konsistent in tiefengleiche Mengen- und Map-Strukturen einfügt.

Listing 44 zeigt diesen Unterschied an einem einfachen Beispiel aus der Modellierung besuchter Roboter-Positionen. In Python kann ein neu konstruiertes Tupel mit demselben Inhalt direkt als bereits bekannt erkannt werden. Dieselbe Eigenschaft stellt in TypeScript erst die Kombination aus `Tuple` und `RecursiveSet` bereit.

```
1 # Python: Tupel sind nativ unveränderlich und inhaltlich vergleichbar
2 visited_coords: set[tuple[int, int]] = set()
3 visited_coords.add((2, 5))
4
5 # Eine Abfrage mit einem neuen, inhaltlich gleichen Tupel gelingt:
6 print((2, 5) in visited_coords) # True
```

```
1 // TypeScript: Tuple stellt das Python-Verhalten für strukturierte Werte her
2 const visitedCoords = new RecursiveSet<Tuple<number, number>>>();
3 visitedCoords.add(new Tuple(2, 5));
4
5 // Eine Abfrage mit einer neuen Tuple-Instanz gelingt ebenfalls:
6 const searchPoint = new Tuple(2, 5);
7 console.log(visitedCoords.has(searchPoint)); // true
```

Listing 44: Roboter-Koordinaten: Tupel in Python vs. TypeScript

3.4 Assoziative Daten: Die `RecursiveMap`

Sobald Koordinaten, Zustände oder andere strukturierte Objekte korrekt modelliert sind, müssen diesen in vielen Algorithmen zusätzliche Werte zugeordnet werden. In der Informatik übernimmt diese Aufgabe eine Map, also eine Datenstruktur, die Schlüssel mit Werten verknüpft. Anschaulich lässt sich dies an einem Schachbrett beschreiben: Einer Koordinate wird eine konkrete Figur zugeordnet. Mathematisch entspricht dies einer Abbildung $f: \text{Schlüssel} \rightarrow \text{Wert}$, etwa $f((2, 5)) = \text{„Queen“}$.

Für die Portierung der Lehrnotebooks genügt es dabei nicht, Schlüssel lediglich als technische Objekte der Laufzeit zu behandeln. Werden Koordinaten oder Zustände als strukturierte Werte modelliert, muss auch ihre Verwendung als Schlüssel nach ihrem inhaltlichen Aufbau erfolgen. Genau an dieser Stelle ergänzt `RecursiveMap` die zuvor eingeführten Klassen `Tuple` und `RecursiveSet`.

Die Klasse `RecursiveMap<K, V>` stellt eine assoziative Datenstruktur bereit, deren

Schlüssel dieselbe fachliche Wertsemantik verwenden wie die übrigen Bibliotheksbausteine. Dadurch können auch neu konstruierte, aber inhaltlich gleiche Schlüssel wiedergefunden werden. Für die Lehrnotebooks entsteht so ein direkt nutzbares Gegenstück zu den in Python üblichen Dictionary-Strukturen, sobald strukturierte Schlüssel benötigt werden.

Listing 45 illustriert dies an einem einfachen Rastermodell. In Python kann ein Koordinatentupel unmittelbar als Schlüssel verwendet werden. In TypeScript übernimmt `RecursiveMap` dieselbe Rolle, sofern die Koordinate durch ein `Tuple` modelliert wird.

```
1 # Python: Tupel als Dictionary-Schlüssel funktionieren direkt
2 grid: dict[tuple[int, int], str] = {}
3 grid[(2, 5)] = "Queen"
4
5 print(grid[(2, 5)]) # "Queen"
```

```
1 // TypeScript: RecursiveMap nutzt Wertsemantik für strukturierte Schlüssel
2 const safeGrid = new RecursiveMap<Tuple<[number, number]>, string>();
3 safeGrid.set(new Tuple(2, 5), "Queen");
4
5 // Der Abruf über eine neue, tiefengleiche Instanz funktioniert ebenfalls:
6 const foundPiece = safeGrid.get(new Tuple(2, 5));
7 console.log(foundPiece); // "Queen"
```

Listing 45: Dictionaries in Python vs. `RecursiveMap` in TypeScript

Wie schon bei `RecursiveSet` liegt der eigentliche Mehrwert damit nicht in einer bloß ähnlichen Oberfläche, sondern in der fachlich korrekten Behandlung strukturierter Daten. Erst dadurch lassen sich Zustände, Positionen oder Übergänge in TypeScript ebenso zuverlässig als Schlüssel verwenden wie in den Python-Originalen.

3.5 Der Freeze-Contract: Immutabilität sichern

Die bisherigen Abschnitte haben gezeigt, dass `RecursiveSet`, `Tuple` und `RecursiveMap` strukturierte Werte fachlich korrekt behandeln. Sobald solche Werte jedoch selbst wieder in größere Mengen- oder Map-Strukturen eingebettet werden, reicht bloße Wertsemantik allein nicht mehr aus. Denn ein Element, das bereits Teil einer übergeordneten Struktur ist, darf seinen inhaltlichen Zustand nachträglich nicht mehr verändern.

Genau an dieser Stelle greift der sogenannte *Freeze-Contract* der Bibliothek. Er legt fest, dass strukturierte Werte ab dem Moment ihrer Einbettung als logisch fixiert gelten. Ein `Tuple`, das als Element einer Menge verwendet wird, und ein `RecursiveSet`, das selbst in einer anderen Menge oder als Schlüssel einer `RecursiveMap` vorkommt, dürfen danach nicht mehr inhaltlich verändert werden.

Für die Benutzung der API bedeutet dies vor allem eines: Sobald ein strukturierter Wert in eine übergeordnete Struktur aufgenommen wurde, ist er als unveränderlich zu behandeln. Damit bleibt sichergestellt, dass Mitgliedschaftstests, Schlüsselzugriffe und Gleichheitsvergleiche auch nach der Einbettung fachlich stabil bleiben.

Listing 46 zeigt dieses Verhalten an einem geschachtelten Beispiel. Die innere Menge kann zunächst normal aufgebaut werden. Nach ihrer Einbettung in eine äußere Menge ist eine weitere Veränderung jedoch nicht mehr zulässig.

```
1 const innerSet = new RecursiveSet<number>(1, 2);
2 const outerSet = new RecursiveSet<RecursiveSet<number>>>();
3
4 outerSet.add(innerSet);
5
6 // Nach der Einbettung ist innerSet logisch fixiert
7 innerSet.add(3); // Fehler
```

Listing 46: Der Freeze-Contract bei geschachtelten Mengen

Das Beispiel verdeutlicht den zentralen Gedanken des Freeze-Contracts: Nicht die Erstellung eines strukturierten Wertes ist eingeschränkt, sondern nur seine nachträgliche Veränderung nach der Einbettung in eine übergeordnete Struktur.

Soll ein solcher Wert dennoch als Ausgangspunkt für weitere Berechnungen dienen, muss zunächst eine eigenständige, wieder veränderliche Kopie, siehe Listing 47 erzeugt werden. Die Bibliothek stellt hierfür mit `mutableCopy()` beziehungsweise `clone()` eine explizite API-Operation bereit. Die anschließende Veränderung betrifft dann nicht mehr das bereits eingebettete Original, sondern ausschließlich die neue Kopie.

```

1  const innerSet = new RecursiveSet<number>(1, 2);
2  const outerSet = new RecursiveSet<RecursiveSet<number>>>();
3
4  outerSet.add(innerSet);
5
6  // Das Original bleibt fixiert
7  const workingCopy = innerSet.mutableCopy(); // alternativ: innerSet.clone()
8  workingCopy.add(3);
9
10 console.log(innerSet);    // {1, 2}
11 console.log(workingCopy); // {1, 2, 3}

```

Listing 47: Arbeiten mit einer veränderlichen Kopie

3.6 Klassische Mengenoperationen

Neben der Konstruktion einzelner Mengen spielen in den Lehrnotebooks vor allem die klassischen Mengenoperationen eine zentrale Rolle. Dazu gehören insbesondere Vereinigung, Schnittmenge, Differenz und symmetrische Differenz. In Python sind diese Operationen direkt in den eingebauten Set-Typ integriert und lassen sich daher sehr kompakt notieren.

Bei der Portierung nach TypeScript entsteht an dieser Stelle ein grundlegender Unterschied in der Ausdrucksweise. Während Python die mathematische Operation teilweise bereits durch sprachnahe Operatoren oder eingebaute Mengenmethoden ausdrückt, muss dieselbe Logik in `RecursiveSet` explizit über API-Aufrufe formuliert werden. Die mathematische Bedeutung bleibt dabei unverändert; lediglich die syntaktische Form wird ausführlicher.

Vereinigung mit `union`

Die Vereinigung zweier Mengen enthält alle Elemente, die in mindestens einer der beiden Ausgangsmengen vorkommen. Doppelte Elemente verschwinden dabei wie üblich aufgrund der Mengensemantik.

```
1 a = {1, 2, 3}
2 b = {3, 4, 5}
3
4 result = a.union(b)
5 print(result) # {1, 2, 3, 4, 5}
```

```
1 const a = new RecursiveSet<number>(1, 2, 3);
2 const b = new RecursiveSet<number>(3, 4, 5);
3
4 const result = a.union(b);
5 console.log(result); // {1, 2, 3, 4, 5}
```

Listing 48: Vereinigung zweier Mengen in Python und mit RecursiveSet

Schnittmenge mit intersection

Die Schnittmenge enthält genau die Elemente, die in beiden Ausgangsmengen gemeinsam vorkommen. Sie eignet sich damit insbesondere für die Bestimmung gemeinsamer Zustände, erreichbarer Konfigurationen oder überlappender Teilmengen.

```
1 a = {1, 2, 3}
2 b = {3, 4, 5}
3
4 result = a.intersection(b)
5 print(result) # {3}
```

```
1 const a = new RecursiveSet<number>(1, 2, 3);
2 const b = new RecursiveSet<number>(3, 4, 5);
3
4 const result = a.intersection(b);
5 console.log(result); // {3}
```

Listing 49: Schnittmenge zweier Mengen in Python und mit RecursiveSet

Differenz mit difference

Die Differenz zweier Mengen enthält alle Elemente der ersten Menge, die nicht in der zweiten Menge vorkommen. Diese Operation ist gerichtet, das heißt $A \setminus B$ ist im Allgemeinen nicht identisch zu $B \setminus A$.

```
1 a = {1, 2, 3}
2 b = {3, 4, 5}
3
4 result = a.difference(b)
5 print(result) # {1, 2}
```

```
1 const a = new RecursiveSet<number>(1, 2, 3);
2 const b = new RecursiveSet<number>(3, 4, 5);
3
4 const result = a.difference(b);
5 console.log(result); // {1, 2}
```

Listing 50: Differenz zweier Mengen in Python und mit RecursiveSet

Symmetrische Differenz mit `symmetricDifference`

Die symmetrische Differenz enthält genau die Elemente, die in einer der beiden Mengen vorkommen, aber nicht in beiden zugleich. Anschaulich lässt sie sich als Vereinigung ohne den gemeinsamen Schnitt verstehen.

```
1 a = {1, 2, 3}
2 b = {3, 4, 5}
3
4 result = a.symmetric_difference(b)
5 print(result) # {1, 2, 4, 5}
```

```
1 const a = new RecursiveSet<number>(1, 2, 3);
2 const b = new RecursiveSet<number>(3, 4, 5);
3
4 const result = a.symmetricDifference(b);
5 console.log(result); // {1, 2, 4, 5}
```

Listing 51: Symmetrische Differenz in Python und mit RecursiveSet

Die Beispiele zeigen, dass die mathematische Bedeutung der Operationen in beiden Sprachen gleich bleibt. Der zentrale Unterschied liegt allein darin, dass Python diese Konzepte bereits als eingebauten Bestandteil seines Set-Typs anbietet, während sie in TypeScript über die explizite API von `RecursiveSet` ausgedrückt werden.

Gerade diese explizite Form hat auch einen Vorteil: Die Operation wird nicht durch eine kompakte Sprachsyntax verborgen, sondern erscheint als klar benann-

ter Methodenaufruf. Damit wird unmittelbar sichtbar, welche Mengenoperation ein Algorithmus gerade tatsächlich ausführt.

3.7 Funktionale Transformationen als Komprehensions-Ersatz

Der weitreichendste sprachliche Unterschied zwischen den Python-Notebooks und der TypeScript-Portierung liegt in der Transformation von Mengen. Python glänzt hier durch sogenannte Mengenkompheensionen (Set Comprehensions), die Selektion und Transformation in einer einzigen, mathematisch äußerst prägnanten Zeile vereinen.

Listing 52 zeigt diesen klassischen Python-Ansatz: Aus einer Menge von Zahlen sollen alle ungeraden Zahlen herausgefiltert und das Ergebnis jeweils mit 10 multipliziert werden.

```
1 mixed_numbers = {1, 2, 3, 4, 5}
2
3 # Die Logik verschmilzt direkt mit der Syntax der Sprache
4 odd_tens = { n * 10 for n in mixed_numbers if n % 2 != 0 }
5
6 print(odd_tens) # {10, 30, 50}
```

Listing 52: Python: Mengenkompheension mit Selektion und Transformation

Da TypeScripts Syntax nicht durch eine externe Bibliothek um derartige Konstrukte erweitert werden kann, muss ein anderes Paradigma genutzt werden: Funktionale Programmierung. Die Bibliothek stellt Methoden bereit, die auf der Menge operieren. Doch wie übergibt man diesen Methoden die individuelle Logik, ohne für jede Kleinigkeit eine eigene benannte Funktion schreiben zu müssen?

Exkurs: Lambda-Funktionen (Arrow Functions)

Die Lösung sind Lambda-Funktionen, in TypeScript „Arrow Functions“ genannt. Eine Arrow Function ist eine anonyme (namenlose) Funktion, die direkt dort definiert wird, wo sie gebraucht wird. Die Syntax lautet `(parameter) => expression`. Der Pfeil `=>` lässt sich am besten als mathematisches „wird abgebildet auf“ lesen. Der Ausdruck `n => n * 10` bedeutet also fachlich exakt: *Nimm das Element n und bilde es auf $n \cdot 10$ ab.*

Mit diesem Werkzeug ausgestattet, können wir die kompakte Python-Kompheension durch die Methode `filterMap` aus `RecursiveSet` nachbauen.

```

1      const mixedNumbers = new RecursiveSet(1, 2, 3, 4, 5);
2
3      const oddTens = mixedNumbers.filterMap(
4          n => n % 2 !== 0,
5          n => n * 10
6      );
7
8      console.log(oddTens.toString()); // {10, 30, 50}

```

Listing 53: TypeScript: Funktionale Pipeline mit `filterMap` und Lambdas

Um diesen Paradigmenwechsel vollständig zu durchdringen, betrachten wir den TypeScript-Code aus Listing 53 Zeile für Zeile:

- **Zeile 3:** Wir rufen die Methode `filterMap` auf unserer Ausgangsmenge auf. Diese Methode iteriert intern über jedes einzelne Element der Menge.
- **Zeile 4:** Dies ist das *Prädikat* (die Filter-Bedingung). Wir übergeben die Lambda-Funktion `n => n % 2 !== 0`. Die Methode wendet diese Funktion auf jedes Element n an. Gibt die Funktion `false` zurück, wird das Element verworfen.
- **Zeile 5:** Dies ist die *Transformation*. Für jedes Element, das den Filter passiert hat, wird nun die zweite Lambda-Funktion `n => n * 10` aufgerufen. Das Ergebnis wird in die neue Zielmenge eingefügt.

Das Ergebnis ist semantisch identisch zum Python-Code, es entsteht jedoch keine einzige unnötige Zwischenmenge im Speicher, da Selektion und Transformation in einem einzigen Durchlauf evaluiert werden.

3.7.1 Basis-Transformationen: `map`, `filter` und `reduce`

3.7.2 Komplexe Konstruktionen: Verschachtelte Iterationen auflösen mit `flatMap`

Noch deutlicher wird die Stärke der funktionalen Lambda-Konzepte, wenn in Python verschachtelte Schleifen (Nested Comprehensions) verwendet werden. Ein typisches Beispiel aus der theoretischen Informatik ist die Berechnung von Folgezuständen. Für jeden aktuellen Zustand muss hierbei oft eine ganze Teilmenge von neuen Zuständen berechnet und in einer gemeinsamen Menge gesammelt werden.

In Python reiht man dafür einfach mehrere `for`-Klauseln hintereinander, wie Listing 54 zeigt.


```

1      integer_set = {0}
2      natural_numbers = [1, 2, 3]
3
4      # Das äußere 'for' läuft über natural_numbers,
5      # das innere (implizite) fügt +n und -n in die Zielmenge ein.
6      expanded_set = { val for n in natural_numbers for val in (n, -n) }
7
8      # integer_set und expanded_set müssen manuell vereinigt werden
9      integer_set |= expanded_set
10     print(integer_set) # {0, 1, -1, 2, -2, 3, -3}

```

Listing 54: Python: Verschachtelte Mengenkomprehension

Für genau diesen Fall der „Eins-zu-Vielen“-Abbildung (ein Eingabewert generiert eine Menge von Ausgabewerten) stellt `RecursiveSet` die Methode `flatMap` bereit. Sie iteriert über eine Eingabe, führt für jedes Element eine Lambda-Funktion aus, die eine Teilmenge zurückgibt, und „flacht“ all diese Teilmengen direkt in die aufrufende Menge ab.

Listing 55 zeigt die direkte TypeScript-Entsprechung.

```

1      const integerSet = new RecursiveSet<number>(0);
2      const naturalNumbers = [1, 2, 3];
3
4      integerSet.flatMap(
5      naturalNumbers,
6      n => new RecursiveSet<number>(n, -n)
7      );
8
9      console.log(integerSet.toString()); // {0, 1, -1, 2, -2, 3, -3}

```

Listing 55: TypeScript: Direkte Generierung und Vereinigung durch `flatMap`

Auch hier hilft eine schrittweise Aufschlüsselung der funktionalen Mechanik:

- **Zeile 4–5:** Wir rufen `flatMap` auf dem bereits existierenden `integerSet` auf und übergeben als Quelle `naturalNumbers`.
- **Zeile 6:** Dies ist die Generator-Logik. Die Lambda-Funktion `n => new RecursiveSet(n, -n)` nimmt sich nacheinander jede natürliche Zahl und erschafft eine temporäre Menge, die den positiven und negativen Wert enthält.
- **Die Abflachung (Flattening):** Die Methode `flatMap` nimmt diese zurückgegebene Teilmenge und fügt deren Inhalt direkt (in-place) in das aufrufende `integerSet` ein.

Zusammenfassend lässt sich festhalten: Die TypeScript-Portierung liest sich aufgrund der fehlenden Comprehensions-Syntax grundlegend anders als das Python-Original. Die mathematische Ausdrucksstärke geht jedoch nicht verloren. Sie wird lediglich von der Syntax-Ebene (dem Compiler) auf die Daten-Ebene verlagert, indem die Algorithmen als kompakte Lambda-Funktionen verpackt und an die API übergeben werden.

4 Vom Python-Werkzeug *PLY* zu *Lezer*

In den ursprünglichen Python-Implementierungen der Lehrveranstaltung werden zentrale Schritte der Sprachverarbeitung mit der Bibliothek *PLY* (Python Lex-Yacc) umgesetzt. *PLY* ist eine Parsergenerator-Bibliothek für Python, die sich an den klassischen Werkzeugen *Lex* und *Yacc* orientiert. Diese Werkzeuge werden häufig in Lehrumgebungen eingesetzt, um grundlegende Konzepte der Compilertechnik wie Tokenisierung und Parsing praktisch umzusetzen. Die Bibliothek wurde von David Beazley entwickelt und ist frei verfügbar [3].

Die grundlegende Architektur eines solchen Sprachprozessors besteht aus zwei zentralen Phasen: der lexikalischen Analyse und der syntaktischen Analyse. Zunächst zerlegt der Lexer den Eingabetext in eine Folge elementarer Token, anschließend analysiert der Parser diese Tokenfolge anhand einer kontextfreien Grammatik und konstruiert daraus eine syntaktische Struktur, etwa einen abstrakten Syntaxbaum (AST) [4, S.5ff].

PLY bildet diese beiden Komponenten explizit ab:

- **Lexikalische Analyse (Lexer):** Token werden über reguläre Ausdrücke definiert. Der Lexer erkennt daraus elementare Spracheinheiten wie Zahlen, Operatoren oder Schlüsselwörter.
- **Syntaxanalyse (Parser):** Die Grammatik der Sprache wird als Menge von Produktionsregeln formuliert. *PLY* generiert daraus einen Parser, der auf LR-Parsing-Techniken basiert. Diese Parsingverfahren lesen die Eingabe von links nach rechts und konstruieren eine rechtsableitende Analyse, wodurch auch komplexe Programmiersprachen effizient verarbeitet werden können.

Ein charakteristisches Merkmal von *PLY* ist, dass Token-Definitionen und Grammatikregeln direkt im Python-Code definiert werden. Produktionsregeln werden dabei als Funktionen formuliert, deren Docstrings die jeweilige Grammatikregel enthalten. Diese enge Integration erleichtert die prototypische Entwicklung von Parsern im Lehrkontext.

Während *PLY* somit eine klassische Compilerarchitektur innerhalb der Python-Umgebung realisiert, verfolgt das TypeScript-Ökosystem einen anderen Ansatz. Für editornahe Anwendungen und webbasierte Entwicklungsumgebungen wird häufig das Parser-Framework *Lezer* eingesetzt.

Lezer verfolgt das Ziel, syntaktische Analysen effizient und inkrementell durchzuführen. Im Gegensatz zu traditionellen Parsergeneratoren wird der Parser so konstruiert, dass er Änderungen am Quelltext lokal verarbeiten kann, ohne den gesamten Syntaxbaum neu berechnen zu müssen [5].

```
1 tokens = ('NUMBER', 'PLUS')
2
3 t_PLUS = r'\+'
4
5 def t_NUMBER(t):
6     r'\d+'
7     t.value = int(t.value)
8     return t
```

Listing 56: Ply-Grammatik in Python

Die Grammatik wird hierbei nicht mehr als Python-Code formuliert, sondern als deklarative Grammatikbeschreibung in einer eigenen Syntax definiert:

```
1 @top Expression { Add }
2
3 Add {
4     Expression "+" Expression
5 }
```

Listing 57: Lezer-Grammatik in TypeScript

Aus dieser Beschreibung generiert *Lezer* einen Parser, der direkt im TypeScript-Ökosystem verwendet werden kann.

Der zentrale Unterschied zwischen beiden Ansätzen liegt somit weniger in der zugrundeliegenden formalen Theorie. Beide basieren auf kontextfreien Grammatiken und klassischen Parsingverfahren, sie unterscheiden sich vielmehr in der Zielumgebung. Während *PLY* primär für klassische Compilerprototypen und Lehrzwecke in Python konzipiert wurde, ist *Lezer* auf moderne Entwicklungswerkzeuge und inkrementelle Analyse von Programmtexten ausgelegt.

Im Kontext dieser Arbeit dient *PLY* daher als Referenzimplementierung der ursprünglichen Python-Notebooks, während *Lezer* die Grundlage für die Umsetzung der gleichen Sprachverarbeitungsschritte im TypeScript-Ökosystem bildet.

4.1 Grundlagen von Lezer

Lezer ist ein Parser-Framework, das im Umfeld des CodeMirror-Projekts entwickelt wurde und speziell auf den Einsatz in modernen Entwicklungswerkzeugen ausgelegt ist [5]. Im Gegensatz zu vielen klassischen Parsergeneratoren steht bei *Lezer* nicht die einmalige Analyse eines vollständigen Programms im Vordergrund, sondern die effiziente Verarbeitung sich kontinuierlich ändernder Programmtexte.

Ein zentrales Ziel von *Lezer* besteht darin, die syntaktische Struktur eines Textes auch während der Bearbeitung im Editor schnell verfügbar zu machen. Funktionen wie Syntax-Highlighting, automatische Einrückung oder Fehlermarkierungen benötigen eine konsistente syntaktische Analyse des aktuellen Dokumentzustands. Da sich der Quelltext während der Bearbeitung ständig verändert, muss der Parser in der Lage sein, diese Änderungen effizient zu verarbeiten.

Inkrementelles Parsing

Eine zentrale Eigenschaft von *Lezer* ist das sogenannte *inkrementelle Parsing*. Dabei wird der Syntaxbaum eines Dokuments nicht bei jeder Änderung vollständig neu berechnet. Stattdessen werden nur jene Teile des Baums aktualisiert, die tatsächlich von der Änderung betroffen sind.

Dieser Ansatz reduziert den Rechenaufwand erheblich und ermöglicht eine nahezu verzögerungsfreie Aktualisierung der Syntaxstruktur im Editor. Besonders bei größeren Dokumenten oder komplexeren Grammatiken stellt dies einen entscheidenden Vorteil gegenüber klassischen Parserarchitekturen dar [6].

Baumrepräsentation der Syntax

Das Ergebnis der Analyse durch *Lezer* ist eine baumartige Repräsentation der syntaktischen Struktur des Eingabetextes. Diese Struktur enthält sämtliche syntaktischen Elemente, die durch die Grammatik definiert wurden.

Lezer erzeugt dabei zunächst einen sogenannten *Concrete Syntax Tree* (CST). Im Gegensatz zu einem abstrakten Syntaxbaum bleiben hier auch Zwischensymbole und strukturelle Details der Grammatik erhalten. Diese vollständige Darstellung der syntaktischen Struktur ist besonders für editornahe Funktionen relevant, da sie eine präzise Lokalisierung einzelner Sprachelemente ermöglicht.

Integration in das JavaScript- und TypeScript-Ökosystem

Ein weiterer wichtiger Aspekt von *Lezer* ist seine Integration in das JavaScript- und TypeScript-Ökosystem. Der generierte Parser kann direkt in Webanwendungen oder Entwicklungswerkzeuge (wie z.B. in Jupyter-Notebook) eingebunden werden.

Im Kontext dieser Arbeit wird *Lezer* verwendet, um die syntaktische Analyse der ursprünglichen Python-Lehrbeispiele in eine TypeScript-basierte Implementierung zu übertragen. Die Grammatik der Sprache bildet dabei die formale Grundlage für die Generierung des Parsers, während TypeScript für die Weiterverarbeitung der analysierten Strukturen eingesetzt wird.

Der nächste Abschnitt beschreibt daher die konkrete Definition der Grammatik, die als Grundlage für die spätere Parsergenerierung dient.

4.2 Grammatikdefinition in Lezer

Die syntaktische Struktur der zu analysierenden Sprache wird in *Lezer* durch eine deklarative Grammatikbeschreibung festgelegt. Diese Grammatik bildet die Grundlage für die spätere Generierung des Parsers und definiert, welche syntaktischen Strukturen innerhalb der Sprache zulässig sind.

Im Rahmen dieser Arbeit wurde die Grammatik direkt innerhalb der verwendeten Jupyter-Notebooks entwickelt. Dieses Vorgehen orientiert sich an der Struktur der ursprünglichen Lehrmaterialien, die ebenfalls notebookbasiert aufgebaut sind. Dadurch konnte die Grammatik schrittweise erweitert und unmittelbar anhand von Beispielausdrücken getestet werden.

Ein wesentliches Merkmal der vorliegenden Implementierung besteht darin, dass die Grammatik nicht in einer separaten Datei abgelegt wurde, sondern als mehrzeiliger String direkt im TypeScript-Code des Notebooks definiert ist. Die Grammatik wird somit als Datenobjekt innerhalb des Programms gehalten und kann ohne Zwischenschritt unmittelbar an die Parsergenerierung übergeben werden. Dieses Vorgehen vereinfacht den experimentellen Umgang mit der Grammatik, da Änderungen direkt an der Stringdefinition vorgenommen und sofort getestet werden können.

Struktur einer Lezer-Grammatik

Eine *Lezer*-Grammatik besteht aus mehreren Komponenten, die gemeinsam die syntaktische Struktur der Sprache beschreiben. Dazu gehören insbesondere:

- die Definition eines Startsymbols,
- Regeln zur Beschreibung komplexer syntaktischer Strukturen,
- sowie Token-Definitionen für elementare Sprachelemente.

Das folgende Beispiel zeigt eine vereinfachte Grammatikdefinition für arithmetische Ausdrücke, wie sie im Notebook als String festgehalten wird:

Die Variable `grammarDefinition` enthält die vollständige Grammatikbeschreibung als Zeichenkette. Durch diese Repräsentation bleibt die Grammatik eng mit dem restlichen Notebook-Code verknüpft. Insbesondere kann sie direkt an die Funktion zur Parsererzeugung übergeben werden, ohne dass ein externer Lade- oder Build-Schritt erforderlich ist.

Das Schlüsselwort `@top` definiert das Startsymbol der Grammatik. Dieses Symbol bildet den Einstiegspunkt der syntaktischen Analyse. Die Regel `Add` beschreibt eine mögliche Struktur eines Ausdrucks: Ein Ausdruck kann entweder aus zwei Teilausdrücken bestehen, die durch einen Additionsoperator verbunden sind, oder aus einem numerischen Wert.

```

1  const grammarDefinition = `
2      @top Expression { Add }
3
4      Add {
5          Expression "+" Expression |
6          Number
7      }
8
9      @tokens {
10         Number { "[0-9]+" }
11     }
12 `;

```

Listing 58: Umsetzung Lezer-Grammatik in TypeScript

Die Definition der Token erfolgt im Block `@tokens`. Hier werden reguläre Ausdrücke verwendet, um elementare Spracheinheiten zu erkennen. Diese Token bilden die Grundlage für die syntaktische Analyse, da sie die kleinsten Einheiten darstellen, aus denen komplexere Strukturen aufgebaut werden.

Iterative Entwicklung der Grammatik

Die Entwicklung der Grammatik erfolgte iterativ innerhalb der Notebook-Umgebung. Nach jeder Anpassung der Stringdefinition konnte der Parser neu generiert und direkt auf Beispielausdrücke angewendet werden. Dieses Vorgehen erleichtert insbesondere die schrittweise Erweiterung der Grammatik, da syntaktische Fehler oder Mehrdeutigkeiten frühzeitig erkannt werden können.

Darüber hinaus unterstützt die Notebook-Umgebung eine experimentelle Arbeitsweise, bei der einzelne Sprachkonstrukte isoliert getestet werden können. Dies ist insbesondere im Lehrkontext hilfreich, da die Auswirkungen einzelner Grammatikregeln unmittelbar sichtbar werden.

Die so definierte Grammatik bildet schließlich die Grundlage für die Generierung des Parsers, die im folgenden Abschnitt beschrieben wird.

4.3 Generierung des Parsers

Nachdem die Grammatik definiert wurde, kann daraus ein ausführbarer Parser erzeugt werden. In klassischen Projekten erfolgt dieser Schritt häufig über ein separates Generatorwerkzeug im Build-Prozess. Im Rahmen dieser Arbeit wurde die Parsergenerierung jedoch direkt innerhalb der Notebook-Umgebung durchgeführt.

Hierzu ist insbesondere das Paket `@lezer/generator` relevant. Nach der offiziellen Dokumentation übernimmt dieses Paket die Erzeugung der Parse-Tabellen aus einer Grammatikbeschreibung. Üblicherweise wird es als Generatorwerkzeug

im Build-Prozess eingesetzt; es kann jedoch auch direkt als JavaScript-Funktion verwendet werden [5]. Genau dieses Vorgehen wurde in der vorliegenden Arbeit genutzt.

Die grundlegende Erstellung des Parsers erfolgt über die Funktion `buildParser`, die aus `@lezer/generator` importiert wird. Diese Funktion nimmt die zuvor definierte Grammatik als Eingabe und erzeugt daraus eine Parserinstanz:

```
1 import { buildParser } from "@lezer/generator";  
2  
3 const parser = buildParser(grammarDefinition);
```

Listing 59: Erstellung des Parsers in TypeScript

Die Variable `grammarDefinition` enthält hierbei die deklarative Beschreibung der Grammatik. Auf Grundlage dieser Definition erzeugt *Lezer* die internen Parsingtabellen und Datenstrukturen, die für die syntaktische Analyse benötigt werden. Der so erzeugte Parser basiert zur Laufzeit auf dem *Lezer-LR*-System, während die erzeugten Syntaxbäume auf den gemeinsamen Baumstrukturen des *Lezer*-Ökosystems beruhen [5].

Parserinstanz

Das Ergebnis des Generierungsprozesses ist ein Parserobjekt, das anschließend zur Analyse von Eingabetexten verwendet werden kann. Der Parser verarbeitet einen Eingabestring und erzeugt daraus eine Baumstruktur, welche die syntaktische Struktur des Ausdrucks repräsentiert.

Diese Struktur ermöglicht es, einzelne Sprachelemente gezielt zu identifizieren und weiterzuverarbeiten. Insbesondere können nachgelagerte Verarbeitungsschritte auf bestimmte Knoten des Syntaxbaums zugreifen, um beispielsweise Ausdrücke auszuwerten oder Transformationen durchzuführen.

Integration in den Notebook-Workflow

Die Generierung des Parsers innerhalb des Notebooks bietet mehrere Vorteile für den Entwicklungsprozess. Änderungen an der Grammatik können unmittelbar getestet werden, da der Parser nach jeder Anpassung neu erzeugt werden kann. Dadurch entsteht ein iterativer Entwicklungsprozess, bei dem Grammatikdefinition und Parserverhalten eng miteinander verknüpft sind.

Dieses Vorgehen entspricht der didaktischen Struktur der ursprünglichen Lehrmaterialien, in denen Sprachkonstrukte schrittweise eingeführt und unmittelbar anhand von Beispielen überprüft werden.

Der generierte Parser bildet schließlich die Grundlage für die syntaktische Analyse innerhalb der TypeScript-Implementierung. Im folgenden Abschnitt wird

beschrieben, wie der Parser in die eigentliche Programmlogik integriert wird und wie die erzeugten Syntaxbäume weiterverarbeitet werden.

4.4 Integration in die TypeScript-Implementierung

Die Generierung des Parsers stellt im Projekt keinen isolierten Endpunkt dar, sondern ist unmittelbar in die weitere TypeScript-Implementierung eingebettet. Der mit `buildParser(grammarDefinition)` erzeugte Parser wird direkt zur Analyse von Eingabestrings verwendet und bildet damit die Eingangsstufe für die weitere Verarbeitung der Sprachelemente.

Im konkreten Projekt greifen dabei mehrere Pakete des *Lezer*-Ökosystems zusammen. Das Paket `@lezer/generator` wird zur Erzeugung des Parsers aus der Grammatikdefinition verwendet. Das Paket `@lezer/common` stellt mit `TreeCursor` eine zentrale Schnittstelle für die Traversierung der erzeugten Baumstruktur bereit. Zusätzlich wird `@lezer/lr` für den LR-basierten Parser-Typ verwendet, sodass die Parserinstanz auch auf Typebene explizit beschrieben werden kann [5].

Parsing von Eingabetexten

Nach der Erzeugung der Parserinstanz kann ein Eingabetext über die Methode `parse` analysiert werden. Diese Methode ist nicht projektspezifisch implementiert, sondern Bestandteil der durch *Lezer* erzeugten Parserinstanz. Im Projekt wird sie über den Ausdruck `parser.parse(input)` aufgerufen. Das Ergebnis ist zunächst keine fachliche Datenstruktur der Anwendung, sondern eine vom Parser erzeugte syntaktische Baumrepräsentation. Diese Struktur enthält die durch die Grammatik erkannten Knoten und dient als unmittelbares Resultat der *Lezer*-Analyse.

Im Notebook wird dieser Schritt direkt mit der anschließenden Weiterverarbeitung verbunden. Ein typisches Muster besteht darin, den Eingabetext zunächst durch den generierten Parser analysieren zu lassen, anschließend einen Cursor auf dem erzeugten Baum zu erzeugen und diesen Baum danach in eine projektspezifische Datenstruktur zu überführen:

Dieses Beispiel verdeutlicht, dass im Projekt zwischen der von *Lezer* bereitgestellten Parsingfunktion und der eigenen Anwendungslogik unterschieden wird. Während `parser.parse(...)` die konkrete Syntax analysiert und einen Syntaxbaum erzeugt, übernimmt die Funktion `parseStmnt` die Kapselung dieses Schritts sowie die anschließende Transformation in eine typisierte interne Repräsentation.

Transformation vom CST zum projektspezifischen AST

Für die eigentliche Programmlogik genügt die konkrete Syntaxstruktur allein nicht. Benötigt wird vielmehr eine Datenstruktur, die auf die Anforderungen der weiteren Verarbeitung abgestimmt ist. Aus diesem Grund wird der durch *Lezer* erzeugte

```

1 import { buildParser } from "@lezer/generator";
2 import { TreeCursor } from "@lezer/common";
3
4 function parseStmnt(input: string): Stmnt {
5     const tree = parser.parse(input);
6     const ast = cstToAST(tree.cursor(), input, config);
7     if (ast instanceof AssignNode || ast instanceof ExprStmntNode) {
8         return ast;
9     }
10    throw new Error(`Expected a statement, got: ${ast.constructor.name}`);
11 }

```

Listing 60: ParseStmnt Funktion in TypeScript

Baum in einen projektspezifischen abstrakten Syntaxbaum (AST) überführt.

Im Projekt erfolgt diese Transformation über die Funktion `cstToAST`. Diese verarbeitet den vom Parser erzeugten Baum rekursiv anhand eines `TreeCursor`. Für jeden Knoten wird geprüft, ob eine zugehörige Transformationsregel definiert ist. Ist dies der Fall, wird aus den Kindknoten ein passender AST-Knoten konstruiert. Knoten, die lediglich syntaktische Zwischenstufen darstellen, können dabei ausgelassen oder automatisch reduziert werden. Die entsprechende Transformationslogik ist im Projekt generisch über `ParserConfig` und `TransformRule` organisiert, sodass die Abbildung vom konkreten Syntaxbaum auf den projektspezifischen AST regelbasiert erfolgt.

Dieses Vorgehen trennt die parsernahe Syntaxrepräsentation klar von der semantisch relevanten internen Modellierung. Die von *Lezer* erzeugte Struktur bleibt damit auf die Analyse des Eingabetexts beschränkt, während die eigentliche Programmlogik ausschließlich mit den definierten TypeScript-Typen und Klassen arbeitet.

Typsicherheit an der Schnittstelle

Ein weiterer Vorteil dieser Integration liegt in der klaren Typisierung der Schnittstelle zwischen Parser und Anwendungscode. Die Funktion `parseStmnt` kapselt nicht nur den Aufruf des generierten Parsers, sondern überprüft zusätzlich, ob das Resultat tatsächlich die erwartete Form besitzt. Dadurch wird verhindert, dass nachgelagerte Verarbeitungsschritte mit ungeeigneten oder unerwarteten Knotenstrukturen arbeiten.

Gerade im Kontext dieser Arbeit ist dieser Schritt von Bedeutung, da die Portierung nach TypeScript nicht nur die syntaktische Analyse reproduzieren soll, sondern zugleich die Vorteile statischer Typisierung nutzbar macht. Die Integration des Parsers endet daher nicht beim Erzeugen eines Syntaxbaums, sondern

umfasst auch dessen kontrollierte Überführung in wohldefinierte, typisierte Datenstrukturen.

Bedeutung für die Gesamtarchitektur

Die Einbettung des *Lezer*-Parsers in die TypeScript-Implementierung zeigt, dass der Parser im Projekt als Teil einer mehrstufigen Verarbeitungskette verstanden wird. Auf die formale Grammatikdefinition folgt die Parsererzeugung, anschließend die Analyse konkreter Eingaben durch den generierten Parser und schließlich die Transformation in eine interne Repräsentation, auf der Auswertung, Analyse oder weitere Übersetzungsschritte aufbauen können.

Damit übernimmt *Lezer* im Gesamtsystem die Aufgabe, die textuelle Eingabe in eine formal strukturierte Zwischenform zu überführen. Die fachliche Bedeutung der erkannten Konstrukte entsteht jedoch erst durch die nachgelagerte Abbildung auf die projektspezifischen AST-Klassen. Genau diese Trennung zwischen syntaktischer Analyse und semantischer Weiterverarbeitung macht die Integration in die TypeScript-Implementierung besonders anschlussfähig und wartbar.

5 Case Study: Umsetzung eines vollständigen Interpreters in TypeScript

In diesem Kapitel wird die praktische Umsetzung der zuvor entwickelten Konzepte anhand des letzten Notebooks (03-Interpreter.ipynb) aus Chapter-07 zur Interpreter-Realisierung dargestellt. Im Mittelpunkt steht nicht nur ein einzelner Ausdrucksparser, sondern eine vollständige Verarbeitungskette für eine kleine c-ähnliche Sprache: von der Grammatikdefinition über die Parsergenerierung und die Transformation des Concrete Syntax Tree (CST) in einen Abstract Syntax Tree (AST) bis hin zur eigentlichen Interpretation des Programms.

Die Fallstudie ist besonders geeignet, um die Unterschiede und Gemeinsamkeiten zwischen der ursprünglichen Python-basierten Lehrumgebung und der TypeScript-Variante sichtbar zu machen. Beide Implementierungen folgen denselben compilertechnischen Grundprinzipien, unterscheiden sich jedoch in der Modellierung der Datenstrukturen, im Zusammenspiel mit dem Typsystem und in der konkreten Werkzeugkette. Der grundsätzliche Ablauf entspricht den klassischen Phasen eines Sprachprozessors: syntaktische Analyse, interne Repräsentation und semantische Ausführung [4, S. 4 ff.].

5.1 Ziel und Aufbau des Notebooks

Das Notebook implementiert einen vollständigen Interpreter für eine kleine imperative Sprache mit Variablen, arithmetischen Ausdrücken, Bedingungen, Schleifen und elementaren Funktionsaufrufen. Die Verarbeitungskette besteht aus vier Schritten:

1. Definition der Sprache durch eine formale Grammatik,
2. Generierung eines Parsers mit Lezer,
3. Umwandlung des erzeugten CST in einen kompakteren AST,
4. rekursive Interpretation des AST.

Diese Struktur entspricht sowohl in der Python- als auch in der TypeScript-Implementierung dem klassischen Aufbau eines Interpreters. Während die Python-Variante typischerweise auf PLY basiert und Lexer sowie Parser explizit trennt, wird in der TypeScript-Version mit Lezer eine integrierte, deklarative Grammatik verwendet. Unabhängig vom eingesetzten Werkzeug bleibt die zugrunde liegende Architektur identisch. Eine formale Grammatik beschreibt die syntaktisch zulässigen Programme, der Parser konstruiert daraus eine Baumdarstellung, und ein

nachgelagerter Verarbeitungsschritt interpretiert diese Repräsentation semantisch [4, S.42ff.].

5.2 Sprachdefinition im Vergleich: Python (PLY) vs. TypeScript (Lezer)

In der ursprünglichen Python-Implementierung wird die Sprache mithilfe von *PLY* (Python Lex-Yacc) definiert. Dabei erfolgt eine klassische Trennung zwischen lexikalischer und syntaktischer Analyse, wie sie in der Compiler-Technik etabliert ist [4, S. 5ff]. Die Tokenisierung wird durch reguläre Ausdrücke beschrieben, während die Grammatikregeln in Form von Python-Funktionen implementiert werden. Dieses Vorgehen orientiert sich eng an traditionellen Werkzeugen wie *Lex* und *Yacc* und spiegelt den zweiphasigen Aufbau eines Compilers wieder.

Dieses Vorgehen führt zu einer engen Verzahnung von Grammatik und Programmlogik. Grammatikregeln sind implizit in Python-Code eingebettet, wodurch die formale Struktur der Sprache weniger explizit sichtbar ist. Gleichzeitig ermöglicht dieser Ansatz eine flexible Integration semantischer Aktionen direkt in den Parser.

Im Gegensatz dazu wird die Sprache in der TypeScript-Implementierung mit *Lezer* deklarativ als Grammatik spezifiziert. Die Grammatik wird dabei als strukturierter String formuliert und anschließend automatisch in einen Parser übersetzt. Dieses Vorgehen entspricht stärker der formalen Definition kontextfreier Grammatiken, wie sie in der Theorie formaler Sprachen üblich sind [7, S. 171ff].

Ein wesentlicher Unterschied zwischen beiden Ansätzen liegt somit im Abstraktionsniveau: Während *PLY* eine imperativ definierte Kombination aus Lexer- und Parserlogik darstellt, beschreibt *Lezer* die Sprache deklarativ auf Basis einer expliziten Grammatik. Diese deklarative Form erleichtert insbesondere die Analyse, Wartbarkeit und Weiterverarbeitung der Grammatik, da die Struktur der Sprache klar getrennt von ihrer Verarbeitung beschrieben wird.

Darüber hinaus unterstützt *Lezer* moderne Anforderungen wie inkrementelles Parsing, das insbesondere für editornähe Anwendungen relevant ist. Der Parser kann dabei Änderungen effizient verarbeiten, ohne die gesamte Eingabe neu analysieren zu müssen. Dies stellt einen wesentlichen Vorteil gegenüber klassischen Parsergeneratoren dar, die typischerweise eine vollständige Neuverarbeitung der Eingabe voraussetzen.

Zusammenfassend lässt sich festhalten, dass *PLY* einen eher traditionellen, eng an der Implementierung orientierten Ansatz verfolgt, während *Lezer* eine stärker formal orientierte und deklarative Beschreibung der Sprache ermöglicht. Diese Unterschiede sind insbesondere im Kontext der Übersetzung von Python

```

1 import ply.lex as lex
2 tokens = [
3     'NUMBER',      # r'0|[1-9][0-9]*'
4     'IDENTIFIER',  # r'[a-zA-Z][a-zA-Z0-9_]*'
5     'ASSIGN',      # r':='
6     'EQ',          # r'=='
7     'NE',          # r'!='
8     'LE',          # r'<='
9     'GE',          # r'>='
10    'IF',           # see below
11    'WHILE'         # see below
12 ]
13 def t_COMMENT(t):
14     r'/*(.\n)*?*/|/\.*/'
15     t.lexer.lineno += t.value.count('\n')
16 def t_NUMBER(t):
17     r'0|[1-9][0-9]*'
18     t.value = int(t.value)
19     return t
20 t_ASSIGN = r':='
21 t_EQ = r'=='
22 t_NE = r'!='
23 t_LE = r'<='
24 t_GE = r'>='
25 Keywords = { 'if': 'IF', 'while': 'WHILE' }
26 def t_IDENTIFIER(t):
27     r'[a-zA-Z][a-zA-Z0-9_]*'
28     t.type = Keywords.get(t.value, 'IDENTIFIER')
29     return t
30 literals = ['+', '-', '*', '/', '%', '(', ')', '{', '}', ';', '<', '>', ',',]
31 t_ignore = ' \t\r'
32 def t_newline(t):
33     r'\n'
34     t.lexer.lineno += 1
35     return
36 def find_column(token):
37     program = token.lexer.lexdata # the complete string given to the scanner
38     line_start = program.rfind('\n', 0, token.lexpos)
39     return token.lexpos - line_start
40 def t_error(t):
41     column = find_column(t)
42     print(f"Illegal character '{t.value[0]}' in line {t.lineno}, column {column}.")
43     t.lexer.skip(1)
44 lexer = lex.lex()

```

Listing 61: Ply-Grammatik

```

1 import { buildParser } from "@lezer/generator";
2 const grammarString = `
3   @top Script { Program }
4   @tokens {
5     Identifier { ${[a-zA-Z] ${[a-zA-Z0-9_]}* }
6     Number      { "0" | ${[1-9] ${[0-9]}* }
7     "+" "_" "*" "/" "%"
8     ":@" "==" "!=" "<=" ">=" "<" ">"
9     "(" ")" "{" "}"
10    ";" " ",
11    space { ${[ \t\n\r]+ }
12    LineComment { "//" ![\n]* }
13    BlockComment { "/*" ( ![*] | "*" + ![*/] )* "*" + "/" }
14    @precedence { LineComment, BlockComment, "/" }
15  }
16  @skip { space | LineComment | BlockComment }
17  IF { @specialize<Identifier, "if"> }
18  WHILE { @specialize<Identifier, "while"> }
19  Program {
20    Stmtnt Program |
21    ""
22  }
23  Stmtnt {
24    IF "(" BoolExpr ")" Stmtnt |
25    WHILE "(" BoolExpr ")" Stmtnt |
26    "{" Program "}" |
27    Identifier ":@" Expr ";" |
28    Expr ";"
29  }
30  BoolExpr {
31    Expr "==" Expr |
32    Expr "!=" Expr |
33    Expr "<=" Expr |
34    Expr ">=" Expr |
35    Expr "<" Expr |
36    Expr ">" Expr
37  }
38  Expr {
39    Expr "+" Product |
40    Expr "-" Product |
41    Product
42  }
`

```

Listing 62: Lezer-Grammatik-Teil1

```

1  ~
2      Product {
3          Product "*" Factor |
4          Product "/" Factor |
5          Product "%" Factor |
6          Factor
7      }
8      Factor {
9          "(" Expr ")"          |
10         Number                |
11         Identifier             |
12         Identifier "(" ExprList ")"
13     }
14     ExprList {
15         NeExprList |
16         ""
17     }
18     NeExprList {
19         Expr |
20         Expr "," NeExprList
21     }
22 ~;
23 const parser = buildParser(grammarString);

```

Listing 63: Lezer-Grammatik-Teil2

nach TypeScript relevant, da sie unterschiedliche Möglichkeiten zur Modellierung und Weiterverarbeitung von Sprachstrukturen bieten.

5.3 Transformation vom CST zum AST

Ein wesentlicher Unterschied zwischen beiden Implementierungen zeigt sich unmittelbar nach dem Parsing. In der Python-Version wird der AST bereits während der Anwendung der Grammatikregeln erzeugt. Die semantischen Aktionen der PLY-Produktionen konstruieren direkt die interne Repräsentation, sodass keine separate Transformationsphase zwischen Parsebaum und abstrakter Syntax mehr erforderlich ist. Die Grammatik und die Konstruktion der Zielstruktur sind damit eng miteinander verbunden.

Die TypeScript-Variante trennt diese Schritte klarer. Lezer erzeugt zunächst einen CST, der anschließend durch eine eigene Transformationsfunktion in einen AST überführt wird. Dadurch wird die klassische Trennung zwischen konkreter und abstrakter Syntax explizit sichtbar [4, S. 42ff.].

Die Python-Variante erzeugt ihre Zielstruktur somit unmittelbar innerhalb der Grammatikregeln, während die TypeScript-Version einen expliziten Zwischenschritt einführt. Diese zusätzliche Transformationsphase erhöht die Modularität der Implementierung und trennt die formale Beschreibung der Sprache deutlicher von ihrer internen Repräsentation.

5.4 Darstellung rekursiver Listenstrukturen

Auch bei der Modellierung rekursiver Listenstrukturen folgen beide Notebooks demselben Grundprinzip. Sowohl Programmlisten als auch Argumentlisten werden als Punktlisten dargestellt. In der Python-Version geschieht dies direkt in den PLY-Regeln.

In der TypeScript-Version bleibt diese Listenstruktur ebenfalls erhalten, wird jedoch explizit durch eine eigene Ausführungsfunktion verarbeitet. Die rekursive Form wird damit nicht nur beim Aufbau des AST, sondern auch bei seiner semantischen Verarbeitung sichtbar.

Die Gemeinsamkeit beider Varianten liegt darin, dass die rekursive Struktur der Grammatik unmittelbar in die interne Repräsentation übernommen wird. Der Unterschied besteht vor allem darin, dass TypeScript diese Listenstruktur zusätzlich durch Hilfsfunktionen systematischer kapselt.

```

1 start = 'program'
2 def p_program_lambda(p):
3     "program : "
4     p[0] = ('.',)
5 def p_program_more(p):
6     "program : stmtnt program"
7     p[0] = ('.', p[1]) + p[2][1:]
8 def p_stmtnt_if(p):
9     "stmtnt : IF '(' bool_expr ')' stmtnt"
10    p[0] = ('if', p[3], p[5])
11
12 def p_stmtnt_while(p):
13     "stmtnt : WHILE '(' bool_expr ')' stmtnt"
14    p[0] = ('while', p[3], p[5])
15
16 def p_stmtnt_block(p):
17     "stmtnt : '{' program '}'"
18    p[0] = p[2]
19
20 def p_stmtnt_assign(p):
21     "stmtnt : IDENTIFIER ASSIGN expr ';' "
22    p[0] = (':=', p[1], p[3])
23
24 def p_stmtnt_expr(p):
25     "stmtnt : expr ';' "
26    p[0] = ('expr', p[1])
27
28 def p_bool_expr_eq(p):
29     "bool_expr : expr EQ expr"
30    p[0] = ('==', p[1], p[3])
31
32 def p_bool_expr_ne(p):
33     "bool_expr : expr NE expr"
34    p[0] = ('!=', p[1], p[3])
35
36 def p_bool_expr_le(p):
37     "bool_expr : expr LE expr"
38    p[0] = ('<=', p[1], p[3])
39

```

Listing 64: Direkte AST-Konstruktion in PLY Teil1

```

1 def p_bool_expr_ge(p):
2     "bool_expr : expr GE expr"
3     p[0] = ('>=', p[1], p[3])
4
5 def p_bool_expr_lt(p):
6     "bool_expr : expr '<' expr"
7     p[0] = ('<', p[1], p[3])
8
9 def p_bool_expr_gt(p):
10    "bool_expr : expr '>' expr"
11    p[0] = ('>', p[1], p[3])
12
13 def p_expr_plus(p):
14    "expr : expr '+' product"
15    p[0] = ('+', p[1], p[3])
16
17 def p_expr_minus(p):
18    "expr : expr '-' product"
19    p[0] = ('-', p[1], p[3])
20
21 def p_expr_product(p):
22    "expr : product"
23    p[0] = p[1]
24
25 def p_product_times(p):
26    "product : product '*' factor"
27    p[0] = ('*', p[1], p[3])
28
29 def p_product_divide(p):
30    "product : product '/' factor"
31    p[0] = ('/', p[1], p[3])
32
33 def p_product_modulo(p):
34    "product : product '%' factor"
35    p[0] = ('%', p[1], p[3])

```

Listing 65: Direkte AST-Konstruktion in PLY Teil2

```

1 def p_product_factor(p):
2     "product : factor"
3     p[0] = p[1]
4
5 def p_factor_paren(p):
6     "factor : '(' expr '"
7     p[0] = p[2]
8
9 def p_factor_number(p):
10    "factor : NUMBER"
11    p[0] = p[1]
12
13 def p_factor_identifier(p):
14    "factor : IDENTIFIER"
15    p[0] = p[1]
16
17 def p_factor_fct_call(p):
18    "factor : IDENTIFIER '(' expr_list '"
19    p[0] = ('call', p[1]) + p[3][1:]
20
21 def p_expr_list_empty(p):
22    "expr_list : "
23    p[0] = ('.',)
24
25 def p_expr_more(p):
26    "expr_list : ne_expr_list"
27    p[0] = p[1]
28
29 def p_ne_expr_list_one(p):
30    "ne_expr_list : expr"
31    p[0] = ('.', p[1])
32
33 def p_ne_expr_list_more(p):
34    "ne_expr_list : expr ',' ne_expr_list"
35    p[0] = ('.', p[1]) + p[3][1:]
36 def p_error(t):
37     column = find_column(t)
38     if t:
39         print(f'Syntax error at token "{t.value}" in line {t.lineno}, column {column}.')
40     else:
41         print('Syntax error at end of input.')
42 parser = yacc.yacc(write_tables=False, debug=True)

```

Listing 66: Direkte AST-Konstruktion in PLY Teil3

```

1 import { cst2ast } from './CST2AST';
2
3 const myOperators = [
4     "IF", "WHILE", ":",
5     "+", "-", "*", "/", "%",
6     "=", "!", "<", ">", "<=", ">="
7 ];
8
9 const myListVars = ["Program", "NeExprList"];
10
11 function parse(fileName: string): AST {
12     const source = readFileSync(fileName, "utf8");
13     const tree = parser.parse(source);
14     return cst2ast(tree.cursor(), source, myOperators, myListVars);
15 }

```

Listing 67: CST-zu-AST-Transformation in TypeScript

```

1 def p_program_lambda(p):
2     "program : "
3     p[0] = ('.',)
4
5 def p_program_more(p):
6     "program : stmt program"
7     p[0] = ('.', p[1]) + p[2][1:]
8
9 def p_expr_list_empty(p):
10    "expr_list : "
11    p[0] = ('.',)
12
13 def p_expr_more(p):
14    "expr_list : ne_expr_list"
15    p[0] = p[1]
16
17 def p_ne_expr_list_one(p):
18    "ne_expr_list : expr"
19    p[0] = ('.', p[1])
20
21 def p_ne_expr_list_more(p):
22    "ne_expr_list : expr ',' ne_expr_list"
23    p[0] = ('.', p[1]) + p[3][1:]

```

Listing 68: Rekursive Listen in der Python-Version

```

1 executeList = (list: AST, env: Environment) => {
2     if (list === "") return;
3
4     if (Array.isArray(list) && list[0] === '.') {
5         const head = list[1];
6         const tail = list[2];
7
8         if (head !== undefined && tail !== undefined) {
9             execute(head, env);
10            executeList(tail, env);
11        }
12    }
13 };

```

Listing 69: Verarbeitung rekursiver Listen in TypeScript

5.5 Typisierung und interne AST-Repräsentation

Ein besonders markanter Unterschied zwischen beiden Implementierungen liegt in der Darstellung des AST. In Python bleibt die Baumstruktur implizit durch Tupelkonventionen beschrieben. Welche Form ein Knoten besitzt, ergibt sich aus dem verwendeten Tag und der Position seiner Operanden. In TypeScript wird dieselbe Struktur dagegen durch einen expliziten Typ beschrieben.

```

1 type Operator = string;
2 type AST = string | number |
3     [Operator, AST] |
4     [Operator, AST, AST] |
5     [Operator, AST, AST, AST] |
6     [Operator, AST, AST, AST, AST];

```

Listing 70: AST-Typdefinition in TypeScript

Diese Typdefinition dokumentiert die rekursive Struktur des AST unmittelbar im Programmtext. Während in Python nur informell klar ist, dass etwa `('+', lhs, rhs)` einen binären Ausdruck repräsentiert, wird in TypeScript explizit festgelegt, dass AST-Knoten aus Zahlen, Bezeichnern oder Operator-Tupeln verschiedener Stelligkeit bestehen. Dadurch wird die Baumstruktur besser dokumentiert und teilweise statisch überprüfbar.

5.6 Interpreterarchitektur im Vergleich

Die Grundstruktur des Interpreters bleibt in beiden Notebooks gleich. Sowohl in Python als auch in TypeScript erfolgt eine Trennung in Anweisungsausführung, arithmetische Auswertung und boolesche Auswertung. In der Python-Version wird dies durch Pattern Matching über Tupel realisiert.

```
1 def execute(stmtnt: NestedTuple, Values: dict[str, Number]) -> None:
2     match stmtnt:
3         case ('.', *SL):
4             execute_tuple(tuple(SL), Values)
5         case (':=', var, value):
6             Values[var] = evaluate(value, Values)
7         case ('expr', expr):
8             evaluate(expr, Values)
9         case ('if', test, stmtnt):
10            if evaluate_bool(test, Values):
11                execute(stmtnt, Values)
12         case ('while', test, stmtnt):
13            while evaluate_bool(test, Values):
14                execute(stmtnt, Values)
15         case _:
16            assert False, f'{stmtnt} unexpected'
```

Listing 71: Ausführung von Anweisungen in Python

Die TypeScript-Version folgt derselben semantischen Idee, verwendet aber explizite Fallunterscheidungen über Knotentags.

Die semantische Struktur bleibt also erhalten, obwohl sich die sprachspezifischen Mittel unterscheiden. Python nutzt Tupel und Pattern Matching, TypeScript verwendet Array-Prüfungen und String-Tags.

5.7 Auswertung arithmetischer Ausdrücke

Auch bei der Auswertung arithmetischer Ausdrücke zeigen sich deutliche strukturelle Parallelen. In Python wird die Semantik direkt durch rekursive Fallunterscheidung über die Tupelform des Knotens beschrieben.

Die TypeScript-Version bildet dieselbe rekursive Struktur nach, ergänzt jedoch explizite Prüfungen auf undefinierte Variablen sowie Fehlerbehandlung bei Division durch Null.

Die prinzipielle Auswertungslogik bleibt damit erhalten. Auffällig ist jedoch, dass die TypeScript-Version die Division mit `Math.floor(l / r)` als ganzzahlige Division realisiert, während in Python direkt der Operator `/` verwendet wird. Damit

```

1 execute = (node: AST, env: Environment): void => {
2     if (!Array.isArray(node)) return;
3     const tag = node[0];
4
5     if (tag === '.') {
6         executeList(node, env);
7         return;
8     }
9
10    if (tag === "block") {
11        const content = node[1];
12        if (content !== undefined) {
13            executeList(content, env);
14        }
15    }
16    else if (tag === ":=") {
17        const name = node[1];
18        const expr = node[2];
19        if (typeof name === "string" && expr !== undefined) {
20            env.set(name, evaluate(expr, env));
21        }
22    }
23    else if (tag === "IF") {
24        const cond = node[1];
25        const body = node[2];
26        if (cond !== undefined && body !== undefined) {
27            if (evaluateBool(cond, env)) execute(body, env);
28        }
29    }
30    else if (tag === "WHILE") {
31        const cond = node[1];
32        const body = node[2];
33        if (cond !== undefined && body !== undefined) {
34            while (evaluateBool(cond, env)) execute(body, env);
35        }
36    }
37    else if (tag === "Call") {
38        evaluate(node, env);
39    }
40 };

```

Listing 72: Ausführung von Anweisungen in TypeScript

```

1 def evaluate(expr: NestedTuple, Values: dict[str, Number]) -> Number:
2     match expr:
3         case int():
4             return expr
5         case str():
6             return Values[expr]
7         case ('call', 'read'):
8             return int(input('Please enter a natural number: '))
9         case ('call', 'print', expr):
10            print(evaluate(expr, Values))
11            return 0;
12        case ('+', lhs, rhs):
13            return evaluate(lhs, Values) + evaluate(rhs, Values)
14        case ('-', lhs, rhs):
15            return evaluate(lhs, Values) - evaluate(rhs, Values)
16        case ('*', lhs, rhs):
17            return evaluate(lhs, Values) * evaluate(rhs, Values)
18        case ('/', lhs, rhs):
19            return evaluate(lhs, Values) / evaluate(rhs, Values)
20        case ('%', lhs, rhs):
21            return evaluate(lhs, Values) % evaluate(rhs, Values)
22        case _:
23            assert False, f'{expr} unexpected'

```

Listing 73: Arithmetische Auswertung in Python

```

1 evaluate = (node: AST, env: Environment): number => {
2     if (typeof node === "number") return node;
3     if (typeof node === "string") {
4         const val = env.get(node);
5         if (val === undefined) throw new Error(`Undefined var ${node}`);
6         return val;
7     }
8
9     if (Array.isArray(node)) {
10        const tag = node[0];
11
12        if (["+", "-", "*", "/", "%"].includes(tag)) {
13            const leftNode = node[1];
14            const rightNode = node[2];
15
16            if (leftNode !== undefined && rightNode !== undefined) {
17                const l = evaluate(leftNode, env);
18                const r = evaluate(rightNode, env);
19                switch(tag) {
20                    case "+": return l + r;
21                    case "-": return l - r;
22                    case "*": return l * r;
23                    case "/":
24                        if (r === 0) throw new Error("Division by zero");
25                        return Math.floor(l / r);
26                    case "%":
27                        if (r === 0) throw new Error("Modulo by zero");
28                        return l % r;
29                }
30            }
31        }
32        if (tag === "Call") {
33            const fn = node[1];
34            const argsNode = node[2];
35
36            if (fn === "print" && argsNode !== undefined) {
37                if (Array.isArray(argsNode) && argsNode[0] === '.') {
38                    const actualArg = argsNode[1];
39                    if (actualArg !== undefined) {
40                        console.log(">> STDOUT:", evaluate(actualArg, env));
41                    }
42                }
            }
        }
    }
}

```

Listing 74: Arithmetische Auswertung in TypeScript Teil 1

zeigt sich, dass eine Portierung nicht nur strukturelle, sondern auch semantische Detailentscheidungen umfasst.

5.8 Auswertung boolescher Bedingungen

Die boolesche Auswertung ist in beiden Implementierungen analog aufgebaut. Vergleichsoperatoren werden rekursiv ausgewertet und liefern einen Wahrheitswert zurück.

```
1 def evaluate_bool(expr: NestedTuple, Values: dict[str, Number]) -> bool:
2     match expr:
3         case ('==', lhs, rhs):
4             return evaluate(lhs, Values) == evaluate(rhs, Values)
5         case ('!=', lhs, rhs):
6             return evaluate(lhs, Values) != evaluate(rhs, Values)
7         case ('<=', lhs, rhs):
8             return evaluate(lhs, Values) <= evaluate(rhs, Values)
9         case ('>=', lhs, rhs):
10            return evaluate(lhs, Values) >= evaluate(rhs, Values)
11         case ('<', lhs, rhs):
12            return evaluate(lhs, Values) < evaluate(rhs, Values)
13         case ('>', lhs, rhs):
14            return evaluate(lhs, Values) > evaluate(rhs, Values)
15         case _:
16            assert False, f'{expr} unexpected'
```

Listing 75: Boolesche Auswertung in Python

Hier zeigt sich besonders deutlich, dass die eigentliche Interpretersemantik weitgehend unverändert bleibt. Unterschiede ergeben sich vor allem aus der konkreten Form der AST-Repräsentation und aus den sprachspezifischen Mitteln der Fallunterscheidung.

5.9 Umgebungsmodell und Programmausführung

Der Programmlauf basiert in beiden Notebooks auf einer Variablenumgebung, die Bezeichner auf numerische Werte abbildet. In Python wird hierfür ein Dictionary verwendet, in TypeScript eine `Map<string, number>`.

Gerade an dieser Stelle wird ein praktischer Unterschied sichtbar: Während die Python-Version interaktive Eingaben direkt über `input(...)` verarbeitet, simuliert die TypeScript-Version Eingaben über einen vorbereiteten `inputStream`. Dies erleichtert insbesondere in Notebook-Umgebungen reproduzierbare Testläufe.

```

1 evaluateBool = (node: AST, env: Environment): boolean => {
2     if (Array.isArray(node)) {
3         const tag = node[0];
4         if (["==", "!=", "<", ">", "<=", ">="].includes(tag)) {
5             const leftNode = node[1];
6             const rightNode = node[2];
7
8             if (leftNode !== undefined && rightNode !== undefined) {
9                 const l = evaluate(leftNode, env);
10                const r = evaluate(rightNode, env);
11                switch(tag) {
12                    case "==": return l == r;
13                    case "!=": return l != r;
14                    case "<":  return l < r;
15                    case ">":  return l > r;
16                    case "<=": return l <= r;
17                    case ">=": return l >= r;
18                }
19            }
20        }
21    }
22    throw new Error("Invalid boolean expression");
23 };

```

Listing 76: Boolesche Auswertung in TypeScript

```

1 def main(file):
2     with open(file, 'r') as handle:
3         program = handle.read()
4         stmtnt = yacc.parse(program)
5         print(stmtnt)
6         Values = {}
7         execute(stmtnt, Values)

```

Listing 77: Programmausführung in Python

```

1 type Environment = Map<string, number>;
2 let inputStream: string[] = [];
3
4 function runProgram(fileName: string, inputs: string[] = []) {
5     console.log(`\n--- Executing File: ${fileName} ---`);
6     console.log(`--- Inputs: [${inputs.join(", ")}] ---`);
7     inputStream = [...inputs];
8     console.log(inputStream);
9     try {
10         const ast = parse(fileName);
11         execute(ast, new Map());
12     } catch (e) { console.error(e); }
13 }

```

Listing 78: Programmausführung in TypeScript

5.10 Ergebnisse der Case Study

Die Gegenüberstellung der restlichen Notebook-Bestandteile bestätigt die zentrale Beobachtung dieser Fallstudie: Die TypeScript-Version übernimmt die grundlegende Interpreterarchitektur des Python-Originals, modelliert die einzelnen Verarbeitungsschritte jedoch expliziter und modularer. Besonders deutlich wird dies an der Trennung von CST und AST, an der typisierten Beschreibung der Baumstruktur sowie an der systematischeren Behandlung rekursiver Listen und Laufzeitfehler.

Gleichzeitig zeigt der Vergleich der konkreten Codeausschnitte, dass die Portierung nicht nur eine Übertragung syntaktischer Konstrukte darstellt. Vielmehr werden bei der Umsetzung in TypeScript Entscheidungen über Abstraktionsniveau, Fehlerbehandlung und teilweise auch über semantische Details getroffen. Die TypeScript-Variante erscheint dadurch als stärker strukturierte und typisierte Rekonstruktion derselben interpretertechnischen Grundidee.

6 Fazit

Ziel dieser Arbeit war die Übertragung der interaktiven Python-basierten Lehrnotebooks der Vorlesung „Theoretische Informatik III“ in das TypeScript-Ökosystem. Dabei wurde bewusst nicht nur eine syntaktische Übersetzung angestrebt, sondern eine strukturelle Neuausrichtung der zugrunde liegenden Architektur. Im Zentrum stand die Frage, inwiefern sich Konzepte formaler Sprachen durch statische Typisierung präziser modellieren und zuverlässiger umsetzen lassen.

Die Ergebnisse der Arbeit zeigen deutlich, dass TypeScript durch sein statisches Typsystem einen wesentlichen Mehrwert für die Modellierung komplexer Datenstrukturen bietet. Während Python viele Fehler erst zur Laufzeit sichtbar macht, ermöglicht TypeScript eine frühzeitige Validierung von Invarianten bereits während der Entwicklung. Insbesondere bei verschachtelten Strukturen wie Syntaxbäumen oder Zustandsmengen trägt dies maßgeblich zur Robustheit der Implementierung bei. Damit wird ein zentraler Vorteil statischer Typisierung bestätigt, nämlich die Vorverlagerung von Fehlererkennung in die Entwicklungsphase [4, S. 371ff].

Ein zentrales Ergebnis der Arbeit ist die Erkenntnis, dass eine naive Portierung nicht ausreicht. Unterschiede in der Laufzeitsemantik, wie etwa das referenzbasierte Verhalten von Mengen in JavaScript, erfordern eigenständige architektonische Lösungen. Die entwickelte Datenstruktur `RecursiveSet` zeigt exemplarisch, wie durch gezielte Implementierungsentscheidungen mathematische Konzepte wie Wertgleichheit und Immutabilität in TypeScript nachgebildet werden können. Gleichzeitig verdeutlicht sie, dass Typsysteme allein nicht alle semantischen Unterschiede zwischen Programmiersprachen überbrücken können, sondern durch geeignete Datenstrukturen ergänzt werden müssen.

Auch im Bereich der Sprachverarbeitung konnte gezeigt werden, dass die grundlegenden Prinzipien der Compilertechnik unabhängig von der Zielplattform bestehen bleiben. Die Transformation von der lexikalischen Analyse über die syntaktische Analyse bis hin zur semantischen Interpretation folgt sowohl in der Python- als auch in der TypeScript-Implementierung denselben theoretischen Grundlagen [4, S. 4ff]. Der Einsatz von Lezer als Parsergenerator verdeutlicht jedoch, wie sich diese klassischen Konzepte an moderne Anforderungen wie inkrementelles Parsing und editornahe Anwendungen anpassen lassen.

Die Case Study zur Implementierung eines vollständigen Interpreters zeigt schließlich, dass sich die gesamte Verarbeitungskette – von der Grammatikdefinition bis zur Ausführung – erfolgreich in TypeScript abbilden lässt. Dabei wird deutlich, dass die statische Typisierung nicht nur der Fehlervermeidung dient, sondern auch die Struktur und Lesbarkeit des Codes verbessert. Gleichzeitig bleibt

ein gewisser Trade-off bestehen: Eine vollständig präzise Typmodellierung kann zu erhöhter Komplexität führen und erfordert bewusste Entwurfsentscheidungen zwischen Strenge und Flexibilität.

Zusammenfassend lässt sich festhalten, dass die Übersetzung der Lehrmaterialien nach TypeScript nicht nur technisch möglich, sondern auch didaktisch sinnvoll ist. Die stärkere Typisierung unterstützt das Verständnis formaler Strukturen, zwingt zur expliziten Modellierung von Invarianten und reduziert typische Fehlerquellen dynamischer Systeme. Gleichzeitig zeigt die Arbeit, dass eine erfolgreiche Migration stets eine konzeptionelle Anpassung erfordert und nicht auf eine reine Syntaxübertragung reduziert werden kann.

Die entwickelten Ansätze bilden damit eine solide Grundlage für zukünftige Erweiterungen, etwa die weitergehende Typisierung von AST-Strukturen oder die Integration zusätzlicher Analyse- und Optimierungsschritte innerhalb der Interpretation.

7 Literaturverzeichnis

- [1] IEEE Computer Society. *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754-2019 (Revision of IEEE 754-2008). IEEE, Juli 2019. URL: https://www-users.cse.umn.edu/~vinals/tspot_files/phys4041/2020/IEEE%20Standard%20754-2019.pdf.
- [2] Georg Cantor. „Beiträge zur Begründung der transfiniten Mengenlehre“. In: *Mathematische Annalen* 46 (1895), S. 481–512. URL: <http://eudml.org/doc/157768>.
- [3] David Beazley. *PLY (Python Lex-Yacc)*. 2026. URL: <https://www.dabeaz.com/ply/> (besucht am 08.02.2026).
- [4] Alfred V. Aho u. a. *Compilers: Principles, Techniques, and Tools*. 2. Aufl. Addison-Wesley, 2006. ISBN: 978-0321486813.
- [5] *Lezer Documentation*. CodeMirror / Lezer Project. 2026. URL: <https://lezer.codemirror.net/docs/> (besucht am 08.02.2026).
- [6] Marijn Haverbeke. *Lezer*. 3. Sep. 2019. URL: <https://marijnhaverbeke.nl/blog/lezer.html> (besucht am 08.02.2026).
- [7] John E. Hopcroft, Rajeev Motwani und Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. 3. Aufl. Pearson, 2006. ISBN: 978-0321455369.

A Anhang

A.1 Verzeichnis der portierten Lehrnotebooks

Die folgende Tabelle listet alle Jupyter-Notebooks auf, die im Rahmen dieser Arbeit selektiert und von Python nach TypeScript portiert wurden. Die Gliederung folgt der Kapitelstruktur der Originalvorlesung „Theoretische Informatik III“.

Tabelle 1: Übersicht der übersetzten Notebooks im Repository

Original-Kapitel	Notebook-Dateiname
Chapter 03	01-Ply-Scanning-Example.ipynb 02-Regex-Tutorial.ipynb 03-Exam-Evaluation.ipynb 04-Html2Text.ipynb
Chapter 04 & 05	01-NFA-2-DFA.ipynb 02-Test-NFA-2-DFA.ipynb 03-Regex-2-NFA.ipynb 04-Test-Regex-2-NFA.ipynb 05-DFA-2-RegExp.ipynb 06-Test-DFA-2-RegExp.ipynb 07-Minimize.ipynb 08-Test-Minimize.ipynb 09-Equivalence.ipynb 10-Test-Equivalence.ipynb
Chapter 06	01-Top-Down-Parser.ipynb 02-EBNF-Parser.ipynb
Chapter 07	01-Calculator.ipynb 02-Differentiator.ipynb 03-Interpreter.ipynb
Chapter 10	01-Conflicts.ipynb 02-Conflicts-Resolved.ipynb

Quelle: Eigene Darstellung.